Université Paris Nanterre

anr agence nationale de la recherche
AU SERVICE DE LA SCIENCE

Master MIAGE

Comparaison expérimentale d'heuristiques pour le problème du vertex cover

0.1 Comparaison expérimentale

Dans cette section, nous présentons une étude expérimentale visant à évaluer les performances de divers algorithmes pour le problème du vertex cover. Nous analyserons leur qualité moyenne, ainsi que leurs performances dans les cas les plus favorables et les moins favorables, en fonction de la taille des solutions obtenues et de leur déviation par rapport à la solution optimale.

Nous commencerons par étudier le rapport d'approximation dans le pire des cas, une mesure couramment employée, qui consiste à comparer directement la valeur de la solution de l'algorithme à celle de la solution optimale dans le scénario le plus défavorable. Par la suite, nous introduirons le rapport d'approximation différentiel qui consiste à calculer la distance de la moyenne des solutions par rapport à la taille de la solution optimale.

Enfin, nous proposerons une validation de l'équiprobabilité des choix aléatoires utilisés dans certaines heuristiques, afin de vérifier que nos résultats expérimentaux ne sont pas influencés par des biais d'échantillonnage.

Le problème du vertex cover appartient à la classe APX, ce qui signifie qu'il admet un algorithme polynomial dont le rapport d'approximation est borné par une constante.

0.1.1 Choix de la mesure de la qualité des solutions

Pour évaluer nos algorithmes dans le cadre du problème du vertex cover, la méthode la plus évidente consisterait à comparer la taille de la solution optimale fournie par chacun de nos algorithmes sur différentes instances. Cependant, cette comparaison seule n'est pas suffisante pour évaluer un algorithme. Prenons l'exemple de l'algorithme *greedy independant cover* (GIC) et de l'algorithme *maximum degree greedy* (MDG). Si le GIC nous fournit une solution de taille 12 et que le MDG fournit une solution de taille 20, il apparaît évident que le GIC offre une solution de meilleure qualité. Cependant, la question subsiste quant à la performance réelle du GIC. Pour répondre à cette interrogation, il est nécessaire de connaître l'optimum de l'instance sur laquelle l'algorithme est testé.

Ainsi, effectivement, le GIC est meilleur, mais si l'optimum est de 2, alors le *rapport d'approximation* du GIC s'écrit

$$\frac{\text{Algo(I)}}{\beta(\text{I})} = \frac{12}{2} = 6$$ [1]

C'est-à-dire que le GIC a produit une solution six fois plus mauvaise que l'optimum. Même s'il dépasse le MDG (12 contre 20), il reste donc considérablement éloigné de l'optimum, permettant de conclure que le GIC n'est pas réellement efficace sur pour cette instance spécifique.

Pour évaluer un algorithme, il est donc nécessaire d'analyser la qualité de la solution retournée par l'algorithme en la comparant à la taille de la solution optimale. Cette approche repose sur le concept du rapport d'approximation *classique*.

Pour nos expériences, nous utiliserons donc comme point de repère la valeur de la solution optimale afin de comparer qualitativement les solutions retournées par les différents algorithmes.

1. Où Algo(I) représente la taille de la solution retournée par l'algorithme sur l'instance I. Et $\beta(I)$ la taille de l'optimum de l'instance I.

Toutefois, il existe des cas particuliers où cette mesure ne reflète pas correctement la qualité d'une solution. Par exemple, dans un graphe complet K_n, la solution optimale pour le Vertex Cover (voir Figure 1) est de taille $n-1$. La *pire solution* consiste à prendre l'ensemble de tous les sommets, soit une taille de n. Or, la différence entre choisir $n-1$ sommets et n sommets devient très faible si l'on regarde uniquement le rapport d'approximation classique car nous obtenons $\frac{n}{n-1} \approx 1 + \frac{1}{n-1}$, ce qui vaut environ 1,01 lorsque n est assez grand. Prendre n sommets donne un *rapport* à peine supérieur à 1, suggérant à tort que l'on est presque optimal. Pourtant, en pratique, choisir n sommets est bien la *pire solution possible*. Il s'agit d'un "comportement anormal" par rapport à la plupart des algorithmes, qui trouvent généralement l'optimum $(n-1)$. Le rapport classique $\frac{n}{n-1}$ ne met donc pas suffisamment en évidence le fait que la solution d'inclure tous les sommets est en réalité la moins efficace qui soit.

Le rapport d'approximation *différentiel* met en évidence cette différence de manière bien plus marquée, car il considère non seulement la solution optimale, mais aussi la *pire solution* faisable notée $w(I)$. Concrètement, pour une instance I, on note :

- $\beta(I)$ la taille minimale d'un vertex cover pour l'instance I.
- w(I) la taille de la pire solution pour l'instance I trouvée par l'algorithme.
- $|S|$ la taille de la solution S trouvée par l'algorithme.

Le rapport différentiel compare alors la portion d'amélioration de l'algorithme par rapport à la pire solution, à l'amélioration maximale obtenue par l'optimum, avec la formule suivante

$$\frac{\mathrm{w}(I) - |S|}{\mathrm{w}(I) - \beta(I)}.$$

Plus ce rapport se rapproche de 1, plus la solution de l'algorithme est proche de l'optimum (car elle a comblé la quasi-totalité de l'écart qui sépare la pire solution de la meilleure). Inversement, un rapport proche de 0 indique une solution peu performante, qui ne s'améliore quasiment pas par rapport à la pire solution.

Le rapport d'approximation différentiel mesure la distance entre nos deux bornes ; l'otimum à la pire solution. Le rapport à l'erreur, au contraire, compare la solution (ou la moyenne des solutions) à la solution optimale, puis normalise cette distance sur l'intervalle $[\beta, w]$. Le rapport à l'erreur est noté $\frac{S(I)-\beta(I)}{w(I)-\beta(I)}$. On peut constater que le rapport d'approximation differentiel et le rapport à l'erreur sont complémentaire et contiennent la même information, une solution proche de l'optimum a un rapport à l'erreur faible, mais un rapport différentiel élevé, et inversement.

$$Pdiff(S) = \frac{\mathrm{w}(I) - |S|}{\mathrm{w}(I) - \beta(I)}$$

$$Perreur(S) = \frac{S(I) - \beta(I)}{w(I) - \beta(I)}$$

Où $Pdiff(S)$ est le rapport d'approximation différentiel et $Perreur(S)$ est le rapport à l'erreur.

On voit facilement que

$$Perreur(S) = 1 - Pdiff(S)$$

Pour notre démarche experimentale, nous exécutons plusieurs fois le même algorithme sur la même instance I. Chaque exécution peut renvoyer une solution différente causé par la part d'aléatoire de certains choix que font les algorithmes par exemple. Pour cela, nous considèrons alors la taille moyenne de ces solutions notée $E(A(I))$. Nous généralisons alors le rapport à l'erreur en remplaçant $S(I)$ par $E(A(I))$.

Pour gérer l'incertitude expérimentale, on discrétise la valeur du rapport à l'erreur en pourcentage, c'est-à-dire qu'on calcule

$$\left\lfloor \frac{E(A(I)) - \beta(I)}{w(I) - \beta(I)} \cdot 100 \right\rfloor$$

L'idée est de regrouper des valeurs très proches de la même classe d'erreur (par exemple, $14,2\%$ et $14,8\%$ deviennent 14%).

Un rapport à l'erreur de 0 signifie que nous avons touché l'optimum tandis qu'un rapport à l'erreur de 1 (soit 100%) signifie que l'algorithme est aussi mauvais que la pire solution.

Dans cette étude, en plus du rapport à l'erreur, nous présenterons le rapport d'approximation en meilleur cas, moyen cas, et en pire cas.

0.2 Démarche expérimentale

Dans cette section, nous présentons notre démarche expérimentale, l'outil utilisé pour la réaliser et les échantillons de graphes sur lesquels nous avons effectués nos tests.

0.2.1 Méthode utilisée

La gestion de l'aléatoire. L'équiprobabilité dans les algorithmes est une propriété essentielle pour garantir l'impartialité des choix lorsque plusieurs options sont disponibles. Dans nos algorithmes, cette propriété signifie que chaque sommet ayant le même degré (maximum ou minimum) doit avoir une probabilité égale d'être sélectionné.

Afin de valider cette caractéristique, nous avons testé le générateur pseudo-aléatoire utilisé (`random.randint` de Python) pour vérifier qu'il produit des tirages uniformément répartis.

La fonction `random.choice` a aussi été testée pour s'assurer que la sélection des sommets parmi ceux ayant le même degré suit une distribution uniforme. Les Figures 1 et 2 illustrent la répartition des sélections pseudo-aléatoires pour des sommets de degré maximal et minimal dans un même graphe. L'expérience a été réalisée sur 10 000 000 de tirages

Dans la Figure 1, chaque sommet de degré maximal a été sélectionné le même nombre de fois.

De même, la Figure 2 montre une répartition similaire pour les sommets de degré minimal, confirmant que l'algorithme ne privilégie aucun sommet en particulier lorsque plusieurs choix équivalents sont possibles.

Cette régularité indique que la fonction `random.choice`, lorsqu'elle est utilisée pour choisir parmi des sommets ayant le même degré, respecte bien l'équiprobabilité attendue.

Propriétés du graphe testé	Valeur
Nombre de sommets	50
Nombre d'arêtes	49
Degré maximal	4
Nombre de sommets de degré maximal	3
Degré minimal	1
Nombre de sommets de degré minimal	18

Table 1 – Propriétés du graphe testé

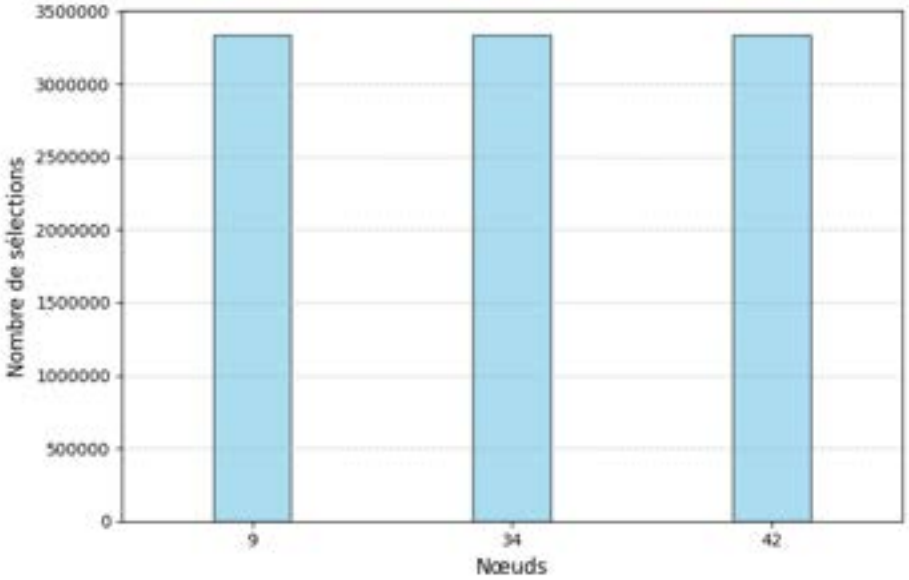

Figure 1 – Répartition des tirages pseudo-aléatoires de trois sommets ayant le même degré maximal dans un graphe.

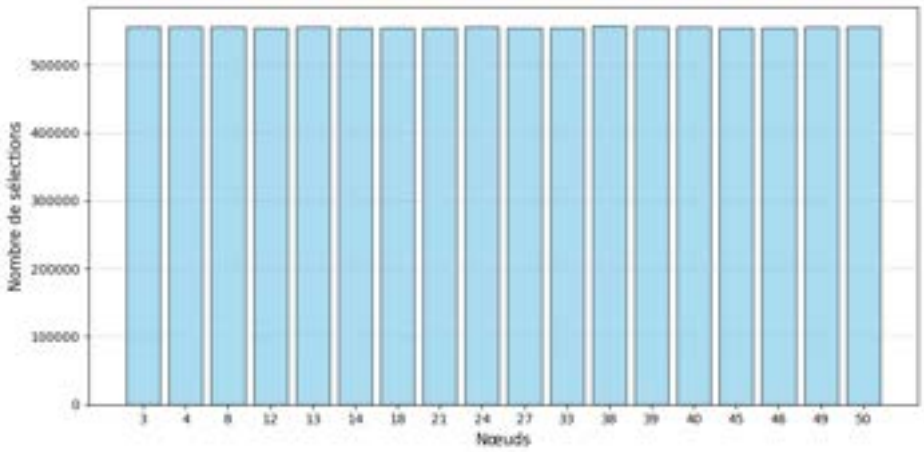

Figure 2 – Répartition des tirages pseudo-aléatoires de trois sommets ayant le même degré minimal dans un graphe.

Pour évaluer l'uniformité du générateur pseudo-aléatoire, nous avons effectué 10 000 000 de tirages aléatoires sur les intervalles, $\{0, 1\}$, $\{0, 10\}$, $\{0, 100\}$ et $\{0, 1000\}$.

Pour chaque intervalle, nous avons collecté les fréquences d'apparition de chaque valeur afin de visualiser sous forme d'histogramme la distribution des fréquences. Un test statistique du Chi^2 a été appliqué pour comparer la distribution observée à une distribution uniforme théorique.

Le test du Chi^2 permet de vérifier l'hypothèse nulle (H_0) selon laquelle les fréquences observées suivent une distribution uniforme. Une p-value supérieure à 0.05 indique que H_0 ne peut pas être rejetée, ce qui suggère que les tirages sont uniformes.

Dans notre cas, la distribution théorique attendue est une loi uniforme discrète sur l'intervalle nos quatre intervalles. Chaque valeur de x dans ces intervalle a une probabilité égale d'être tirée.

Par exemple, si X suit une distribution uniforme discrète sur $\{0, 10\}$, alors :

$$P(X = x) = \frac{1}{11}, \quad \text{pour } x \in \{0, 1, 2, \ldots, 10\}$$

Nous avons effectué $N = 10^7$ tirages. La fréquence attendue pour chaque valeur de x est donc :

$$E = N \times P(X = x) = 10^7 \times \frac{1}{11} \approx 909090.91$$

Ce qui signifie que si `random.randint(0,10)` suit bien une distribution uniforme, chaque nombre de 0 à 10 devrait apparaître environ 909 091 fois, avec de légères variations dues à l'aléatoire.

Les résultats des tests pour chaque intervalle sont présentés dans le tableau 2 ci-dessous.

Pour tous les intervalles testés, les p-values obtenues sont généralement supérieures au seuil de 0.05, ce qui suggère que les tirages suivent une distribution uniforme. Le générateur aléatoire maintient une distribution uniforme, indépendamment de la taille de l'intervalle.

Intervalle	Chi2 Stat	p-value
$\{0, 10\}$	16.98	0.0747
$\{0, 1\}$	0.042	0.837
$\{0, 100\}$	79.97	0.930
$\{0, 1000\}$	987.67	0.603

Table 2 – Résultats du test Chi2 pour les différents intervalles

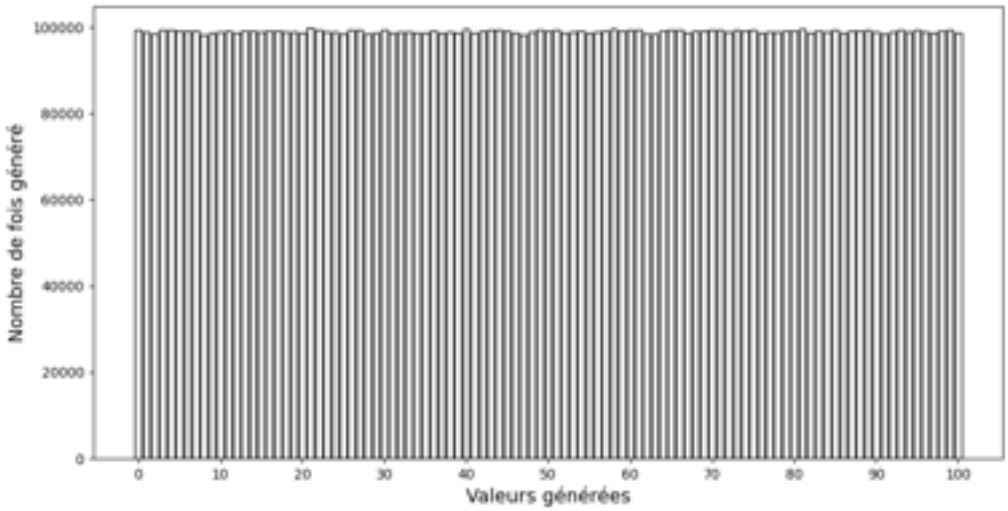

FIGURE 3 – Répartition des tirages pseudo-aléatoires pour l'intervalle [0,100].

Déroulement d'une expérience. Tout d'abord, nous avons déterminé la liste algorithmes à comparer, puis la liste des échantillons, c'est-à-dire les ensembles d'instances de graphes à tester au cours de notre expérience avec la taille de la solution optimale. Pour calculer la taille d'une solution optimale pour chaque instance, nous avons utilisé la programmation linéaire en nombres entiers. Pour chaque graphe généré, nous avons utilisé la librairie python *PULP* pour trouver une solution optimale avec le solveur CBC (Coin-or Branch and Cut). Nous avons récupéré certains de nos échantillons disponible sur internet, qui nous fournissent la taille de la solution optimale, par exemple les graphes BHOSLIB [2].

Le principal défi résidait dans la taille conséquente des graphes générés. Le problème du vertex cover étant un problème NP-complet, il était difficile de résoudre efficacement les instances de grande taille. Certains graphes atteignaient jusqu'à dix milles sommets.

Pour cette raison, j'ai commencé par générer des graphes de petite taill pour chaque graphe. Ensuite, j'ai appliqué le solveur directement sur ces graphes, et tant qu'une solution optimale était trouvée, la génération des graphes continuait tout en modifiant progressivement les paramètres de construction afin d'obtenir des graphes plus grands et plus denses.

Un fichier au format `DIMACS`, contenant l'instance du graphe, était créé si et seulement si le solveur trouvait la taille de la solution optimale. Cela se vérifait par les différents statuts que retourne le solveur (`LPStatus`) :

— `Not Solved`
— `Optimal`
— `Infeasible`
— `Unbounded`
— `Undefined`

Ainsi, nous avons généré dix instances pour chaque taille.

2. Benchmark BHOSLIB avec solution pour le vertex cover

0.2.2 Nos échantillons de tests

Les graphes réguliers Nous avons généré des graphes réguliers où chaque sommet possède exactement le même degré. Plus précisément, nous avons considéré des tailles de graphes $n \in \{28, 40, 52, 64, 76, 98\}$ et des degrés $d \in \{3, 4, 5, 6, 7, 8\}$. Pour chaque couple (n, d), un graphe d-régulier a été généré de manière aléatoire en assurant la reproductibilité via une graine fixe.

Les graphes de Erdős-Rényi Nous avons généré des graphes aléatoires selon le modèle d'Erdős-Rényi $G(n, p)$. Pour chaque taille de graphe $n \in \{20, 40, 60, 80, 100\}$ et pour chaque probabilité d'arête $p \in \{0.1, 0.2, 0.3, 0.4\}$, nous avons généré plusieurs instances indépendantes.

Les graphes en arbre Nous avons généré des graphes en arbre à l'aide d'une approche basée sur un graphe complet initial. La procédure suivie est la suivante :

1. On commence avec un graphe complet de $n > 3$ sommets.
2. On retire aléatoirement une arête tout en maintenant la connectivité du graphe.
3. L'étape précédente est répétée jusqu'à ce que le graphe contienne exactement $n - 1$ arêtes, garantissant ainsi qu'il forme un arbre.

Nous avons utilisé cette méthode pour générer des arbres de tailles $n \in \{50, 100, 200, 350\}$ avec plusieurs instances par taille, en assurant la reproductibilité grâce à une graine fixe.

BHOSLIB Nous avons également inclus les benchmarks BHOSLIB, qui sont des graphes difficiles issus de la transformation d'instances SAT satisfiables. Ces graphes sont générés selon le modèle RB et ont été récupérés depuis le site officiel BHOSLIB. Nous avons testé nos algorithmes sur cinq instances de 450 sommets (30 cliques) avec une solution optimale connue de 420 sommets, bien que la composition exacte de cette solution ne soit pas fournie.

0.2.3 Les algorithmes utilisés

Dans cette section, nous présentons les algorithmes d'approximation et heuristiques que nous avons implémentés et utilisés dans notre étude comparative. Notre objectif est d'évaluer leur efficacité en fonction de la qualité des solutions retournées sur différentes classes de graphes. Nous comparerons leurs performances en utilisant les métriques introduites précédemment, en particulier le rapport à l'erreur.

Les algorithmes que nous avons étudiés couvrent une large gamme de stratégies, allant des approches classiques basées sur le degré des sommets, aux techniques plus avancées intégrant des principes d'optimisation locale et métaheuristiques. Nous avons sélectionné tous ces algorithmes pour leur diversité.

Nous distinguons trois grandes catégories d'algorithmes :

— **Les heuristiques basées sur le degré des sommets.**
— **Les algorithmes basés sur les appariements et les transformations de graphe.**
— **Les méthodes d'optimisation locale et métaheuristiques.**

Heuristiques basées sur le degré des sommets

Ces algorithmes reposent sur des stratégies de sélection des sommets en fonction de leur degré et s'appuient souvent sur des règles gloutonnes.

- **Maximum Degree Greedy (MDG)** : Sélectionne itérativement les sommets de degré maximal jusqu'à couvrir toutes les arêtes.
- **Minimum Degree Greedy (MinDG)** : Sélectionne itérativement les sommets de degré minimal, une approche généralement moins efficace que MDG.
- **Sorted List Left (SLF) & Sorted List Right (SLR)** : Appliquent une sélection ordonnée des sommets selon leur degré dans l'ordre croissant ou décroissant.
- **Degree Reduction** : Privilégie la sélection des sommets en tenant compte des réductions successives du degré dans le graphe.
- **Edge Deletion** : Sélectionne les sommets de manière à éliminer progressivement les arêtes jusqu'à obtenir un vertex cover.

Algorithmes basés sur les appariements et les transformations de graphe

Ces méthodes exploitent des propriétés structurelles du graphe pour construire une solution approchée.

- **Matching Based** : Utilise un appariement maximal comme base pour construire un vertex cover.
- **Approximate Matching** : Applique une heuristique pour construire rapidement un appariement approximatif avant de générer la solution.
- **Bar-Yehuda Even** : Algorithme basé sur une relaxation linéaire du problème, suivi d'un arrondi pour obtenir une solution admissible.
- **Primal-Dual** : Méthode d'optimisation inspirée de la programmation linéaire et de la théorie des dualités.

Algorithmes d'optimisation locale et métaheuristiques

Ces algorithmes tentent d'améliorer une solution initiale à travers des ajustements successifs.

- **Local Search** : Améliore une solution en effectuant des modifications locales pour réduire la taille du vertex cover.
- **Local Search Ratio** : Variante de la recherche locale intégrant un critère d'amélioration basé sur un ratio de gain.
- **Iterated Local Search (ILS)** : Effectue une recherche locale répétée avec des perturbations pour éviter les minima locaux.
- **Variable Neighborhood Search (VNS)** : Explore différentes structures de voisinage pour améliorer progressivement la solution.
- **Depth-First Search (DFS)** : Implémente une exploration récursive pour construire des solutions en profondeur.

Neuf autres algorithmes ont été codés pour élargir nos comparaisons, tel que l'algorithme génétique ou encore le BFS. Ces algorithmes font l'objet de différents dysfonctionnements, c'est pourquoi nous ne les avons pas introduit dans nos expériences.

0.2.4 Résultats

Les résultats sont regroupés par échantillons, à raison d'un échantillon réparti sur deux pages. Chaque page contient 8 figures qui représentent les performances de chaque algorithme. L'abscisse représente le pourcentage d'erreur obtenu par l'algorithme pour une instance de l'échantillon et l'ordonnée représente le nombre d'instances de l'échantillon ayant obtenu ce rapport à l'erreur.

Dans la suite, nous dirons que l'algorithme A domine l'algorithme B sur une instance I lorsque le rapport à l'erreur de l'algorithme A sur l'instance I est inférieur à celui de l'algorithme B. Il faut bien garder en mémoire qu'une telle relation de dominance ne signifie pas que toute exécution de l'algorithme A est meilleure que celle de B, mais qu'il s'agit d'un résultat obtenu en moyenne après 300 itérations de chaque algorithme, ce qui donne une tendance générale des solutions retournées.

Les figures 4 à 19 sont obtenues en appliquant les algorithmes sur l'échantillon du benchmark BHOSLIB.

Les figures 20 à 35 sont obtenues en appliquant les algorithmes sur l'échantillon de graphes de Erdös-Renyi.

Les figures 36 à 52 sont obtenues en appliquant les algorithmes sur l'échantillon de graphes régulier.

Les figures 53 à 69 sont obtenues en appliquant les algorithmes sur l'échantillon des arbres.

Échantillons	AOC	Approximate Matching	Bar Yehuda Even	Degree Reduction	ED
Erdös-Rényi	$0-100$	$0-85$	$40-100$	$0-100$	$0-83$
BHOSLIB	$100-100$	$81-85$	96	$100-100$	$81-85$
Régulier	$42-83$	$40-75$	$63-81$	$0-100$	$40-76$
Arbres	$33-80$	$37-75$	$69-87$	$100-100$	$37-75$

Table 3 – Tableau récapitulatif des intervalles de pourcentages d'erreur obtenus pour chaque algorithme pour chaque échantillon. Le premier nombre représente le meilleur pourcentage d'erreur obtenu sur l'échantillon et le deuxième eprésente le plus mauvais (Partie 1)

Échantillons	DFS	GIC	Local Search	Local Search Ratio	Matching Based	ILS
Erdös-Rényi	N/A	$0-100$	$0-100$	$0-83$	$100-100$	$0-100$
BHOSLIB	$100-100$	$100-100$	$88-100$	$81-85$	$100-100$	N/A
Régulier	$100-100$	$0-50$	$0-71$	$40-76$	$100-100$	$0-71$
Arbres	$100-100$	0	$0-66$	$33-78$	$100-100$	$0-66$

Table 4 – Tableau récapitulatif des intervalles de pourcentages d'erreur obtenus pour chaque algorithme pour chaque échantillon. (Partie 2)

Échantillons	Max Degree Greedy	Min Degree Greedy	Primal Dual	SLF	SLR	VNS
Erdös-Rényi	$0-100$	$100-100$	$100-100$	$0-100$	$50-100$	$0-66$
BHOSLIB	$72-100$	$100-100$	$100-100$	$95-100$	$92-100$	$81-85$
Régulier	$0-62$	$100-100$	$100-100$	$50-80$	$50-76$	$25-63$
Arbres	$0-62$	$100-100$	$100-100$	$42-77$	$78-96$	$22-63$

Table 5 – Tableau récapitulatif des intervalles de pourcentages d'erreur (Partie 3)

note : Faire une analyse sur les graphes des echantillons (voir page 93).

Échantillons	1	2	3	4	5	6	7	8
Erdös-Rényi	VNS	ED	LSR	AM	AOC	DR	GIC	LS
BHOSLIB	MDG	LS	AM	ED	LRS	VNS	SLR	SLF
Régulier	GIC	LS	MDG	ILS	DR	VNS	AM	ED
Arbres	GIC	LS	MDG	ILS	VNS	AOC	LRS	SLF

Table 6 – Classement corrigé des algorithmes par échantillon (Partie 1)

Échantillons	9	10	11	12	13	14	15	16
Erdös-Rényi	SLF	ILS	MDG	BE	MinDG	PDual	MB	
BHOSLIB	BE	PDual	DR	MinDG	MB	AOC	GIC	DFS
Régulier	LRS	SLF	SLR	PDual	MinDG	MB	BE	DFS
Arbres	SLR	ED	BE	PDual	MinDG	MB	DR	DFS

Table 7 – Classement corrigé des algorithmes par échantillon (Partie 2)

Légende des acronymes :
— AM : Approximate Matching
— BE : Bar Yehuda Even
— DR : Degree Reduction
— ED : Edge Deletion
— LS : Local Search
— LRS : Local Search Ratio
— MB : Matching-Based
— MDG : Maximum Degree Greedy
— MinDG : Minimum Degree Greedy
— PDual : Primal Dual
— SLF : Sorted ListLeft
— SLR : Sorted ListRight
note : Faire une constatation des résultats (voir page 94)

Rapport d'approximation à différentiel

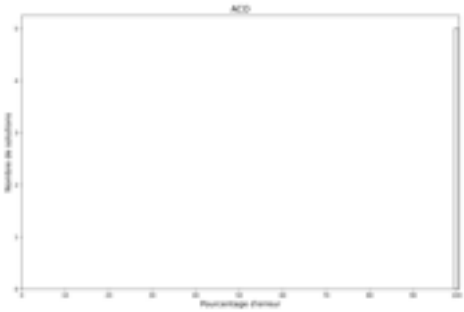

FIGURE 4 – ACO

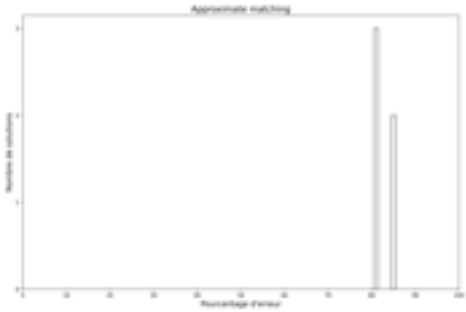

FIGURE 5 – Approximate Matching

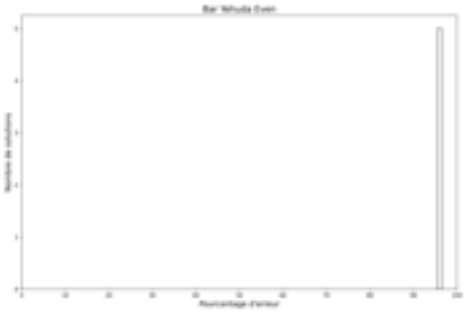

FIGURE 6 – Bar Yehuda Even

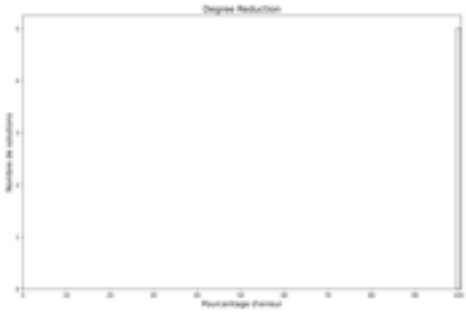

FIGURE 7 – Degree Reduction

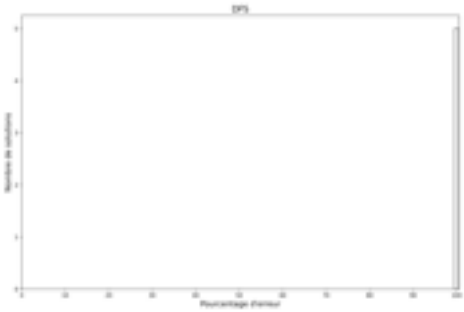

FIGURE 8 – DFS

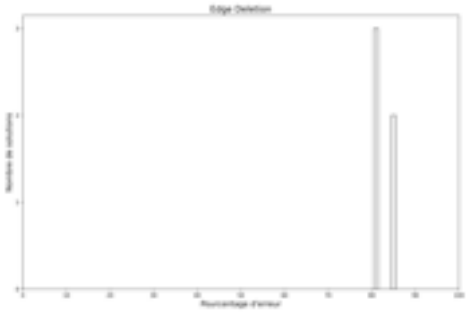

FIGURE 9 – Edge Deletion

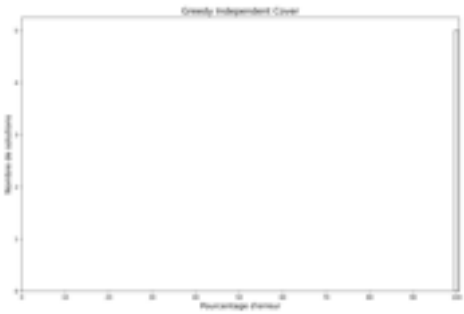

FIGURE 10 – Greedy Independant Cover

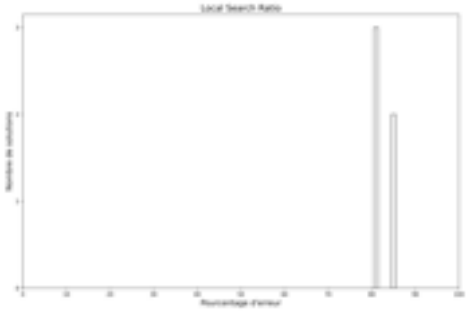

FIGURE 11 – Local Search Ratio

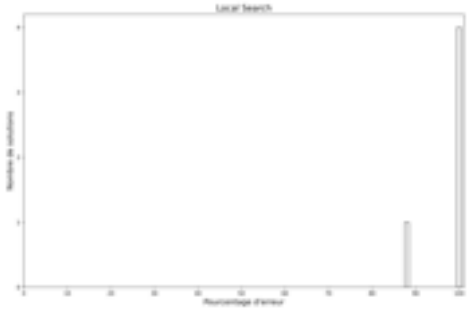

FIGURE 12 – Local Search

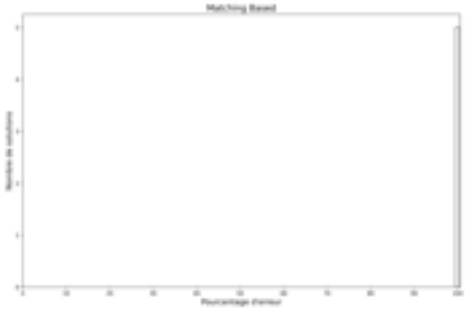

FIGURE 13 – Matching Based

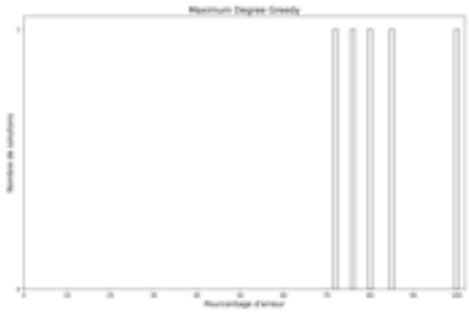

FIGURE 14 – Maximum Degree Greedy

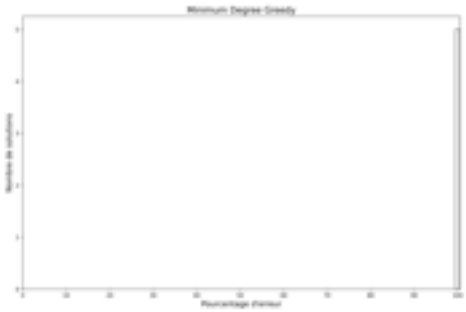

FIGURE 15 – Minimum Degree Greedy

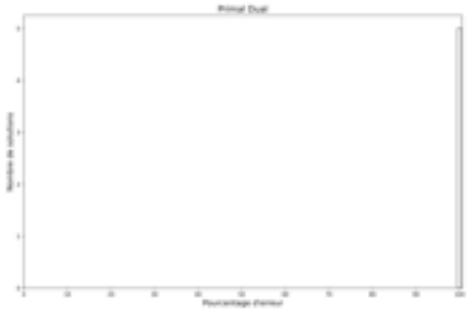

FIGURE 16 – Primal Dual

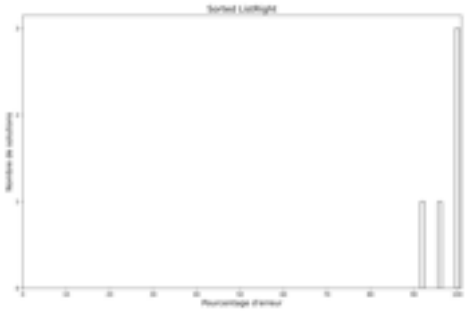

FIGURE 17 – Sorted ListRight

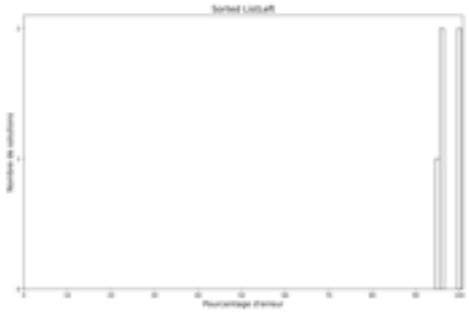

FIGURE 18 – SortedListLeft

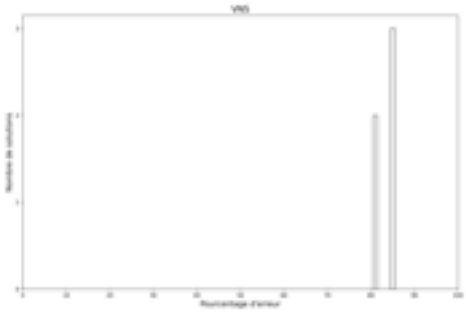

FIGURE 19 – VNS

Échantillons du benchmark BHOSLIB

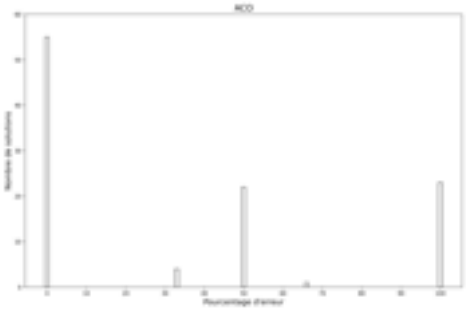

Figure 20 – ACO

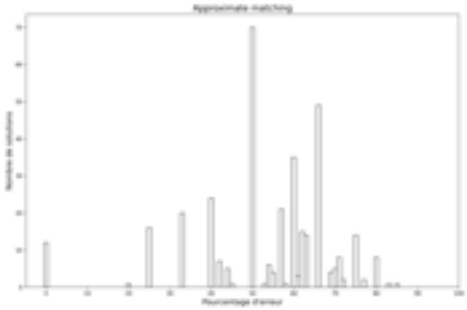

Figure 21 – Approximate Matching

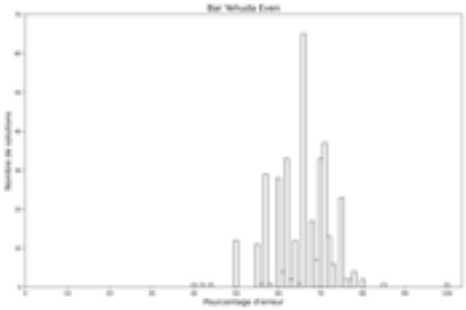

Figure 22 – Bar Yehuda Even

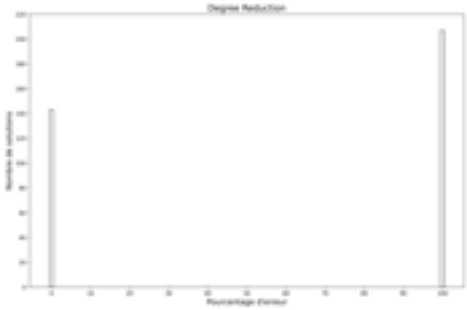

Figure 23 – Degree Reduction

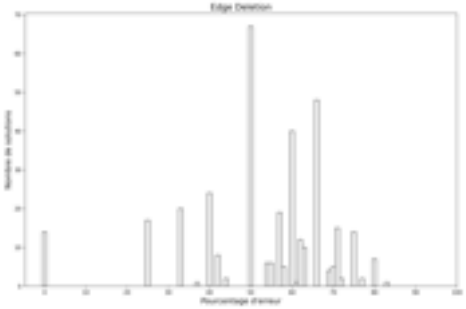

Figure 24 – Edge Deletion

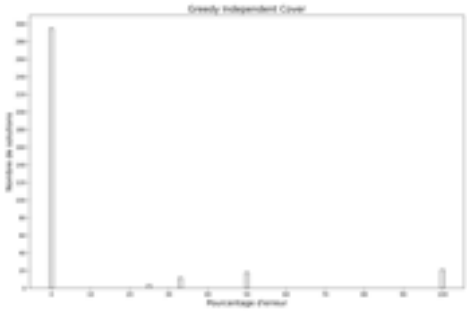

Figure 25 – Greedy Independant Cover

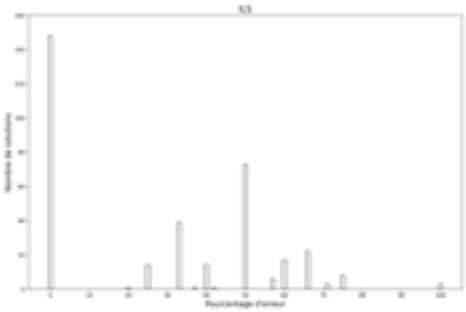

Figure 26 – ILS

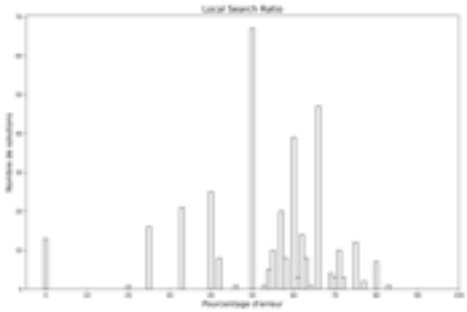

Figure 27 – Local Search Ratio

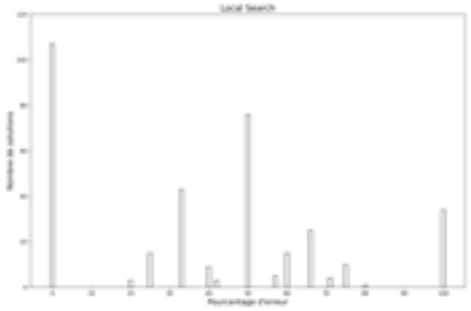

FIGURE 28 – Local Search

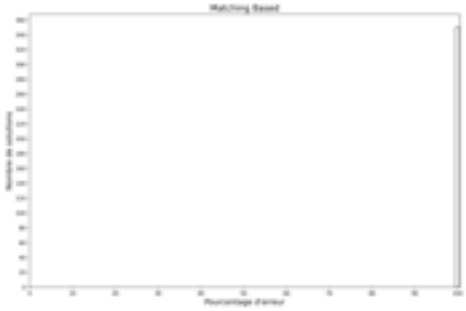

FIGURE 29 – Matching Based

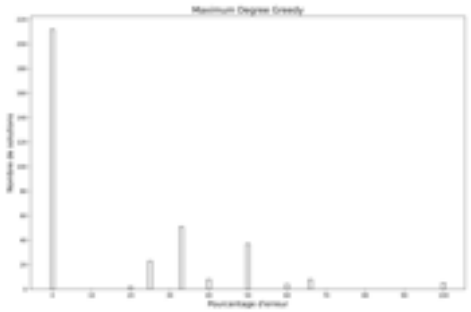

FIGURE 30 – Maximum Degree Greedy

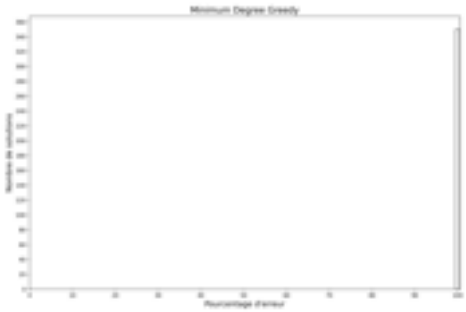

FIGURE 31 – Minimum Degree Greedy

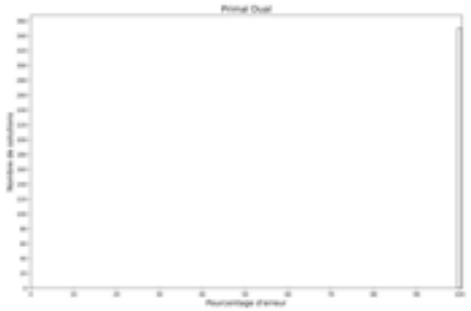

FIGURE 32 – Primal Dual

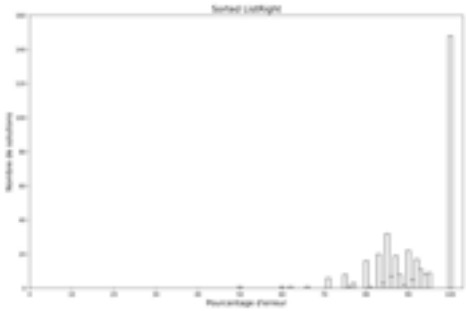

FIGURE 33 – Sorted ListRight

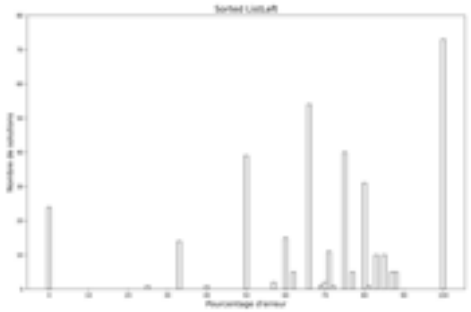

FIGURE 34 – Sorted ListLeft

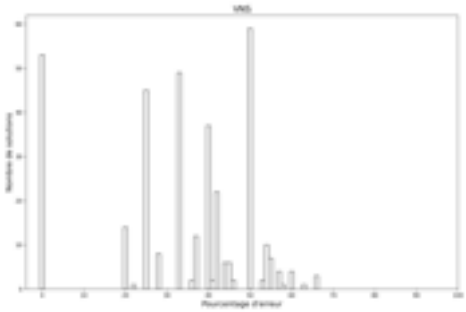

FIGURE 35 – VNS

Échantillons des graphes de Erdös-Renyi

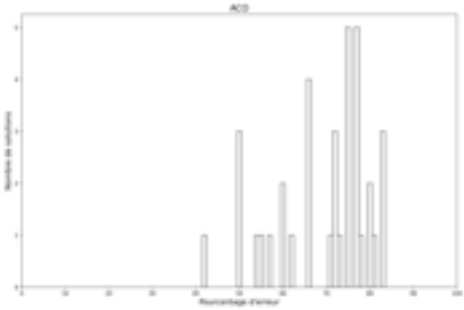

Figure 36 – ACO

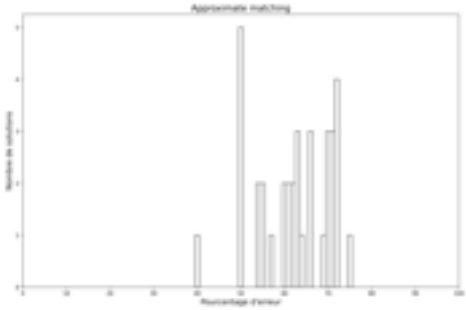

Figure 37 – Approximate Matching

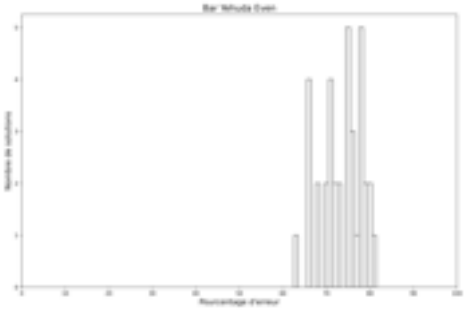

Figure 38 – Bar Yehuda Even

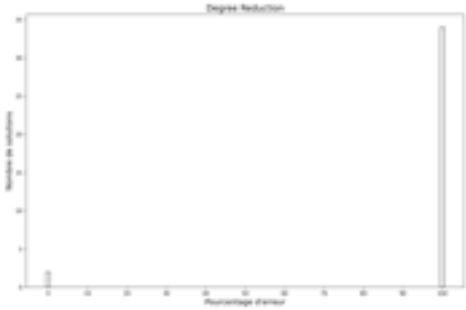

Figure 39 – Degree Reduction

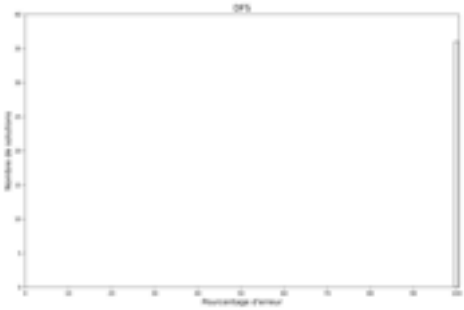

Figure 40 – DFS

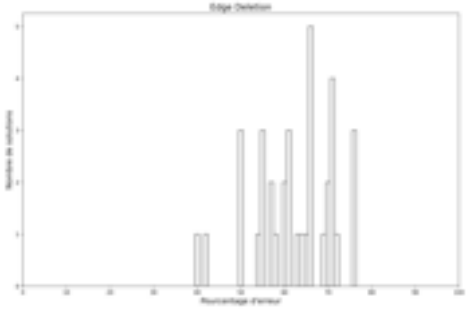

Figure 41 – Edge Deletion

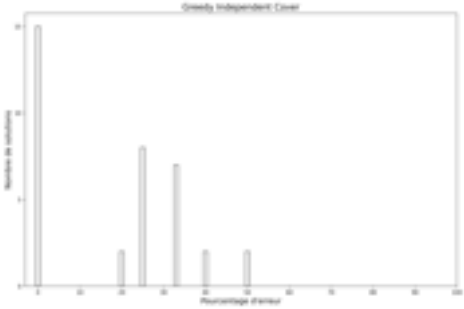

Figure 42 – Greedy Independant Cover

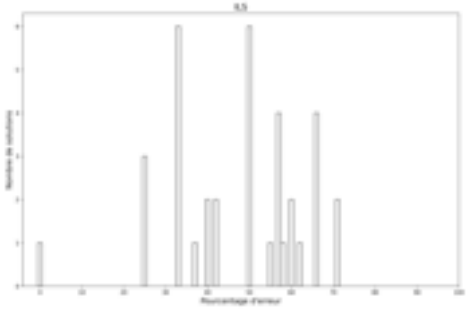

Figure 43 – ILS

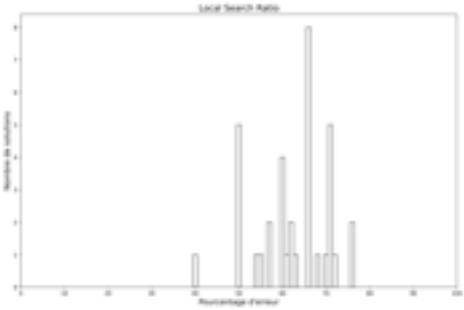

FIGURE 44 – Local Search Ratio

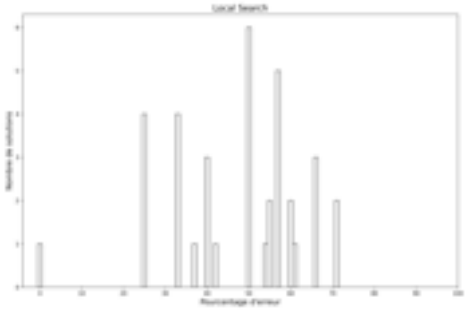

FIGURE 45 – Local Search

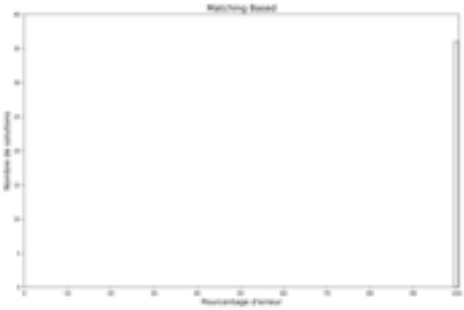

FIGURE 46 – Matching Based

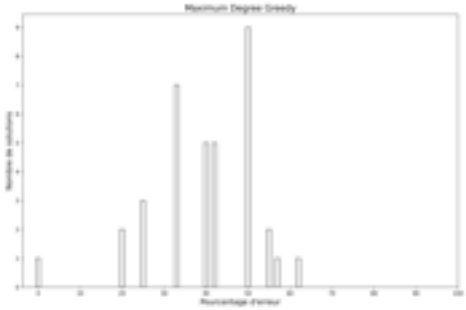

FIGURE 47 – Maximum Degree Greedy

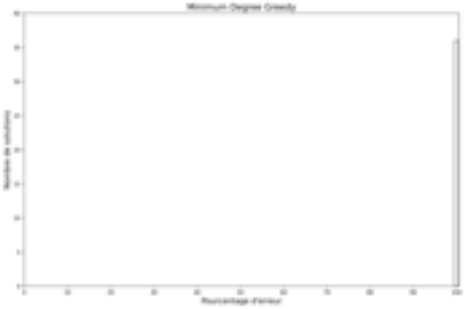

FIGURE 48 – Minimum Degree Greedy

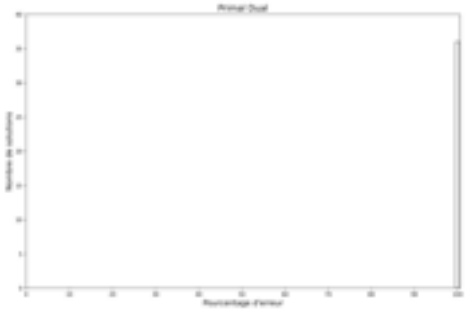

FIGURE 49 – Primal Dual

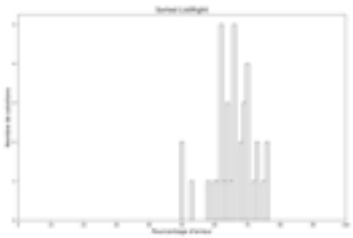

FIGURE 50 – Sorted ListRight

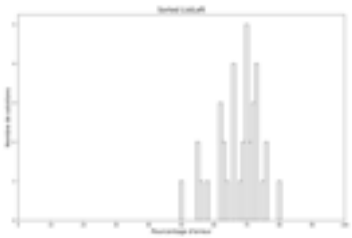

FIGURE 51 – Sorted ListLeft

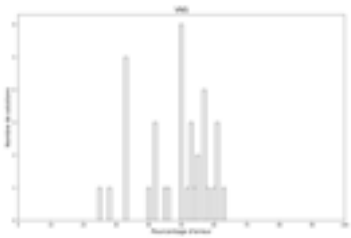

FIGURE 52 – VNS

Échantillons des graphes régulier

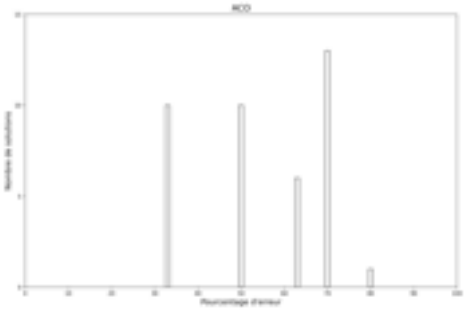

Figure 53 – ACO

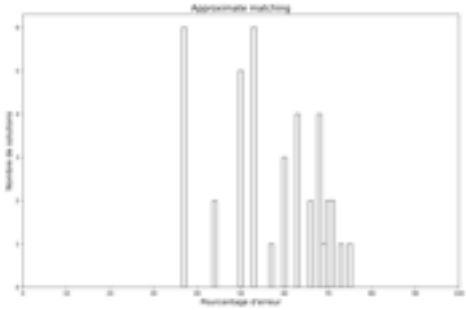

Figure 54 – Approximate Matching

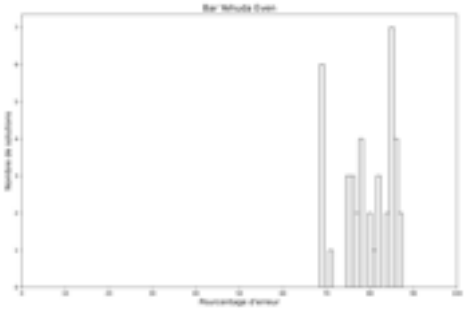

Figure 55 – Bar Yehuda Even

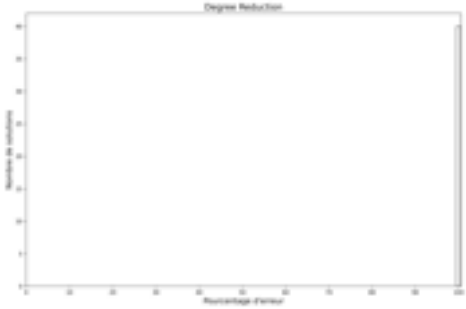

Figure 56 – Degree Reduction

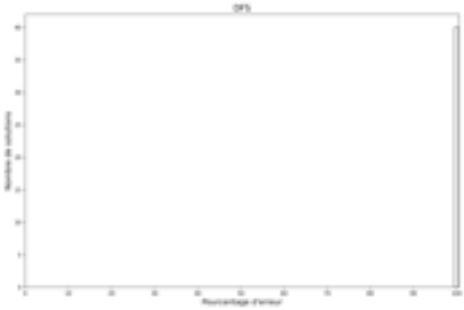

Figure 57 – DFS

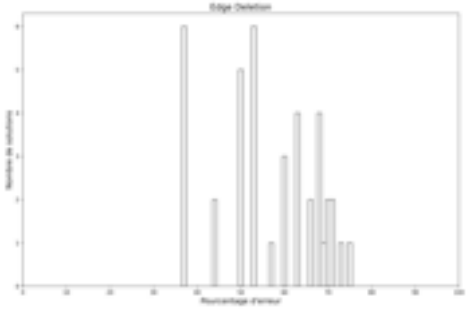

Figure 58 – Edge Deletion

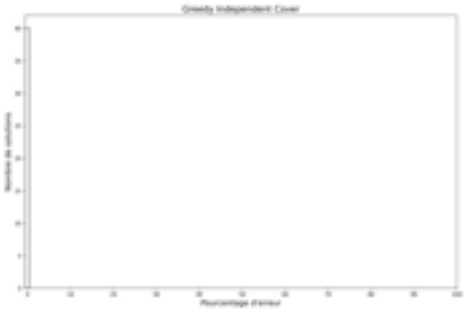

Figure 59 – Greedy Independant Cover

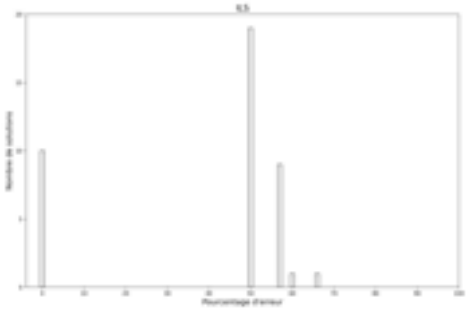

Figure 60 – ILS

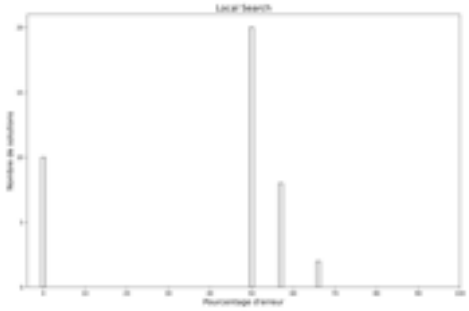

Figure 61 – Local Search Ratio

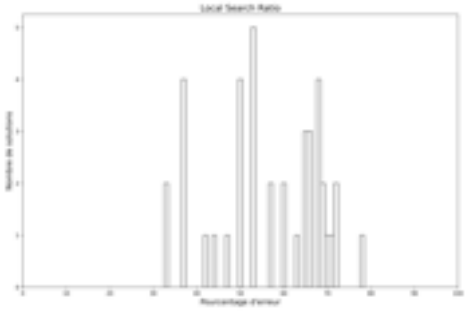

Figure 62 – Local Search

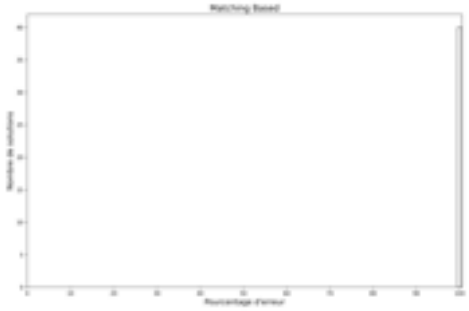

Figure 63 – Matching Based

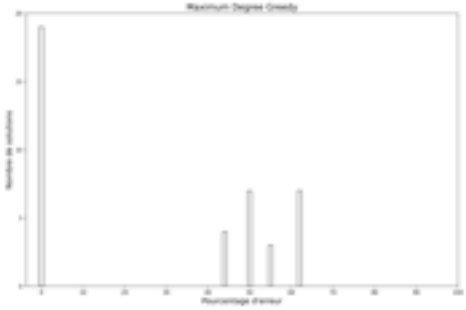

Figure 64 – Maxium Degree Greedy

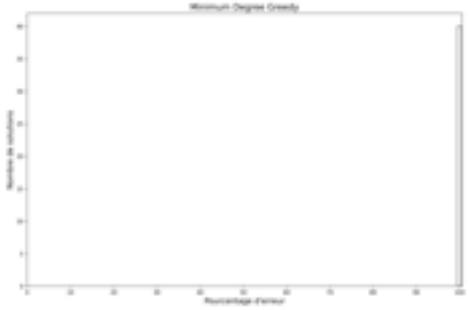

Figure 65 – Minimum Degree Greedy

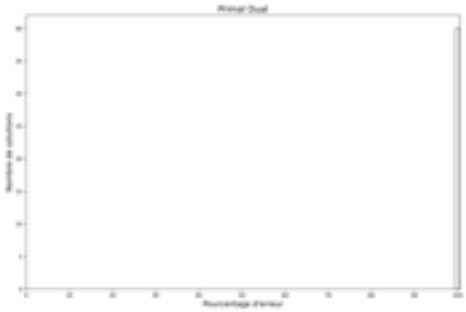

Figure 66 – Primal Dual

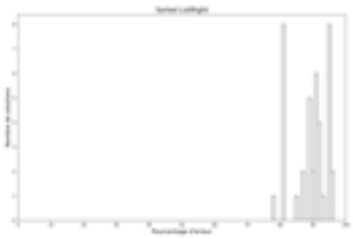

Figure 67 – Sorted ListRight

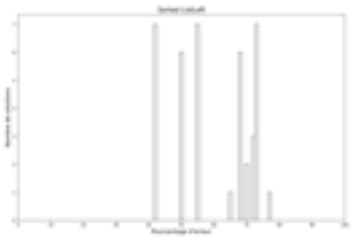

Figure 68 – Sorted ListRight

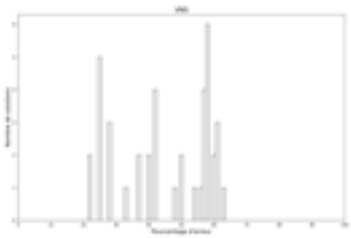

Figure 69 – VNS

Échantillons des arbres

0.2.5 Bilan

Bibliographie

[label] Auteur, TITRE, editeur, annee

[LAM94] L. LAMPORT, *LaTeX : A Document preparation system, Addison-Wesley, 1994*

Webographie

[CAT] `savoircoder.fr/cat`

Chapitre 1

Annexes

Annexe .1 : Illustration du vertex cover sur un graphe complet

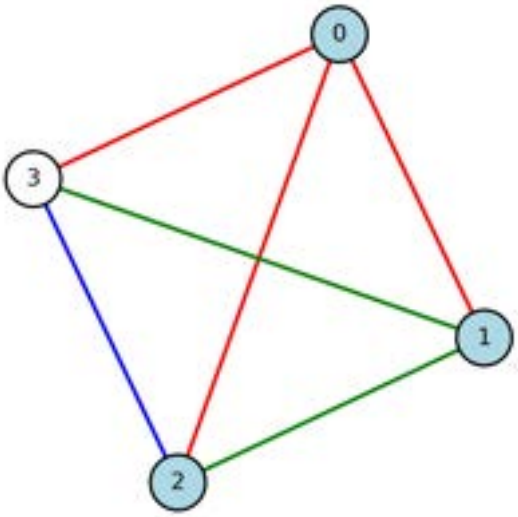

FIGURE 1 – Représentation du vertex cover minimal pour le graphe complet K_4. Les sommets sélectionnés dans la couverture sont colorés. Chaque arête est colorée en fonction du sommet qui la couvre (rouge, vert et bleu). Le sommet blanc est exclu du vertex cover.

Annexe .2 : Pseudo-codes des algorithmes

Annexe .3 : Résultats des algorithmes sur différentes classes de graphes

Annexe .3.1 : Résultats en meilleur cas

Échantillons	AOC	Approximate Matching	Bar Yehuda Even	Degree Reduction	ED
Erdös-Rényi	$0-20$	$0-27$	$0-100$	$0-18$	$0-27$
BHOSLIB	7	3	6	3	3
Régulier	$0-23$	$0-18$	$4-42$	$0-14$	$0-19$
Arbres	$0-7$	$0-11$	$39-61$	$2-5$	$0-11$

Table 1 – Tableau récapitulatif des intervalles de pourcentages d'erreur obtenus pour chaque algorithme pour chaque échantillon. Le premier nombre représente le meilleur pourcentage d'erreur obtenu sur l'échantillon et le deuxième eprésente le plus mauvais (Partie 1)

Échantillons	DFS	GIC	Local Search	Local Search Ratio	Matching Based	ILS
Erdös-Rényi	N/A	$0-8$	$0-27$	$0-27$	$9-100$	$0-16$
BHOSLIB	$5-6$	3	3	3	$5-6$	N/A
Régulier	$14-59$	0	$0-11$	$0-16$	$23-72$	$0-11$
Arbres	$31-40$	0	$0-3$	$0-11$	$71-91$	$0-3$

Table 2 – Tableau récapitulatif des intervalles de pourcentages d'erreur obtenus pour chaque algorithme pour chaque échantillon. (Partie 2)

Échantillons	Max Degree Greedy	Min Degree Greedy	Primal Dual	SLF	SLR	VNS
Erdös-Rényi	$0-14$	$14-280$	$9-100$	$0-30$	$11-100$	$0-20$
BHOSLIB	3	6	$5-6$	$5-6$	6	3
Régulier	$0-4$	$28-78$	$23-72$	$4-24$	$4-22$	$0-9$
Arbres	$0-2$	$100-135$	$71-91$	$4-11$	$65-98$	$0-3$

Table 3 – Tableau récapitulatif des intervalles de pourcentages d'erreur obtenus pour chaque algorithme pour chaque échantillon. (Partie 3)

Annexe .3.2 : Résultats en moyen cas

Échantillons	AOC	Approximate Matching	Bar Yehuda Even	Degree Reduction	ED
Erdös-Rényi	$0-20$	$0-30$	$11-100$	$0-18$	$0-30$
BHOSLIB	7	4	6	3	4
Régulier	$9-26$	$9-28$	$23-57$	$0-14$	$9-28$
Arbres	$2-7$	$13-19$	$65-73$	$2-5$	$3-19$

Table 4 – Tableau récapitulatif des intervalles de pourcentages d'erreur obtenus pour chaque algorithme pour chaque échantillon. Le premier nombre représente le meilleur pourcentage d'erreur obtenu sur l'échantillon et le deuxième eprésente le plus mauvais (Partie 1)

Échantillons	DFS	GIC	Local Search	Local Search Ratio	Matching Based	ILS
Erdös-Rényi	N/A	$0-8$	$0-27$	$0-30$	$9-100$	$0-18$
BHOSLIB	$5-6$	3	3	4	$5-6$	N/A
Régulier	$14-59$	$0-4$	$0-13$	$9-28$	$23-72$	$0-13$
Arbres	$31-40$	0	$0-3$	$13-19$	$71-91$	$0-3$

Table 5 – Tableau récapitulatif des intervalles de pourcentages d'erreur obtenus pour chaque algorithme pour chaque échantillon. (Partie 2)

Échantillons	Max Degree Greedy	Min Degree Greedy	Primal Dual	SLF	SLR	VNS
Erdös-Rényi	$0-14$	$14-280$	$9-100$	$0-33$	$11-120$	$0-27$
BHOSLIB	$3-4$	6	$5-6$	$5-6$	$6-6$	4
Régulier	$0-11$	$28-78$	$23-72$	$14-36$	$14-36$	$4-23$
Arbres	$0-4$	$113-135$	$71-91$	$8-15$	$78-102$	$8-16$

Table 6 – Tableau récapitulatif des intervalles de pourcentages d'erreur obtenus pour chaque algorithme pour chaque échantillon. (Partie 3)

Annexe .3.3 : Résultats en pire cas

Échantillons	AOC	Approximate Matching	Bar Yehuda Even	Degree Reduction	ED
Erdös-Rényi	$0-30$	$6-80$	$17-100$	$0-18$	$6-71$
BHOSLIB	7	5	7	3	5
Régulier	$18-36$	$21-50$	$33-76$	$0-14$	$23-50$
Arbres	$4-13$	$22-39$	$81-100$	$2-5$	$22-39$

Table 7 – Tableau récapitulatif des intervalles de pourcentages d'erreur obtenus pour chaque algorithme pour chaque échantillon. Le premier nombre représente le meilleur pourcentage d'erreur obtenu sur l'échantillon et le deuxième eprésente le plus mauvais (Partie 1)

Échantillons	DFS	GIC	Local Search	Local Search Ratio	Matching Based	ILS
Erdös-Rényi	N/A	$0-20$	$0-37$	$6-71$	$9-100$	$0-37$
BHOSLIB	$5-6$	3	$3-4$	5	$5-6$	N/A
Régulier	$14-59$	$4-11$	$8-27$	$23-50$	$23-72$	$4-27$
Arbres	$31-40$	0	$0-6$	$21-40$	$71-91$	$0-6$

Table 8 – Tableau récapitulatif des intervalles de pourcentages d'erreur obtenus pour chaque algorithme pour chaque échantillon. (Partie 2)

Échantillons	Max Degree Greedy	Min Degree Greedy	Primal Dual	SLF	SLR	VNS
Erdös-Rényi	$0-30$	$14-280$	$9-100$	$0-60$	$11-180$	$6-62$
BHOSLIB	$3-5$	6	$5-6$	$5-6$	6	5
Régulier	$9-25$	$28-78$	$23-72$	$28-56$	$28-54$	$19-45$
Arbres	$5-10$	$113-135$	$71-91$	$12-30$	$91-114$	$22-42$

Table 9 – Tableau récapitulatif des intervalles de pourcentages d'erreur obtenus pour chaque algorithme pour chaque échantillon. (Partie 3)

Annexe .4 : Rapport d'approximation en meilleur cas

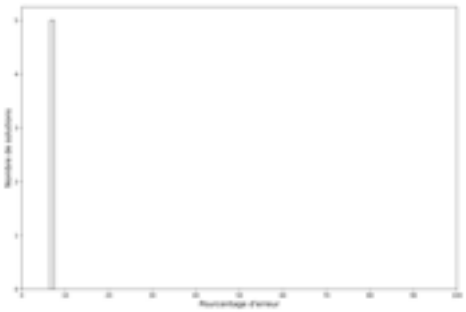

Figure 2 – ACO

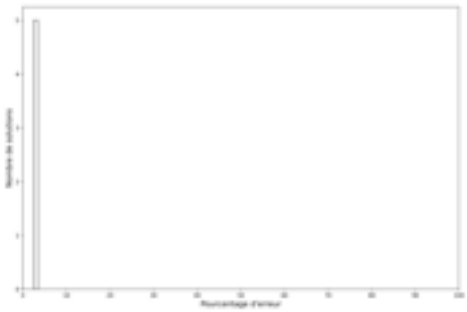

Figure 3 – Approximate Matching

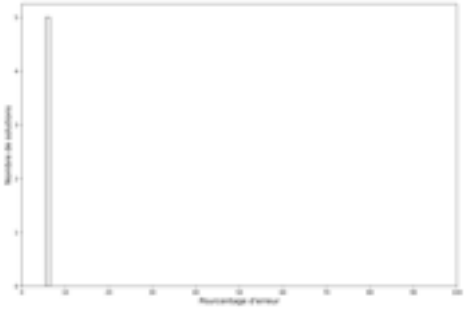

Figure 4 – Bar Yehuda Even

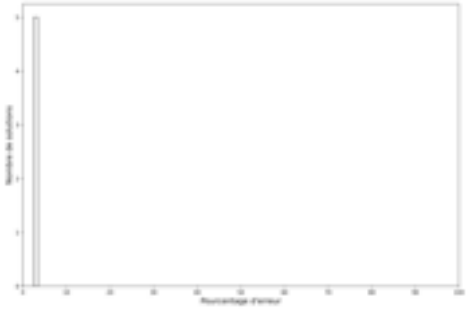

Figure 5 – Degree Reduction

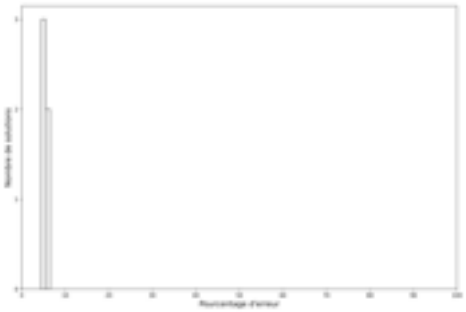

Figure 6 – DFS

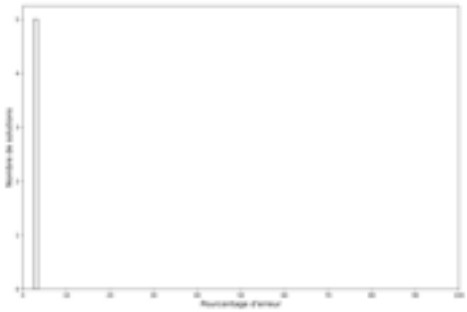

Figure 7 – Edge Deletion

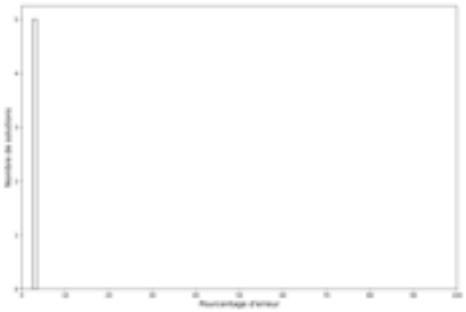

Figure 8 – Greedy Independant Cover

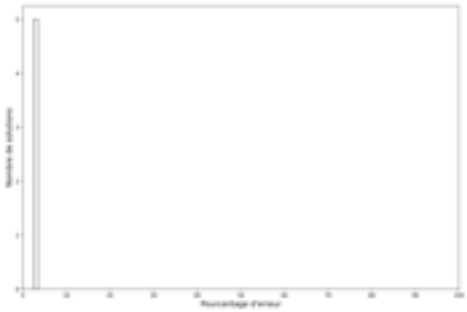

Figure 9 – Local Search Ratio

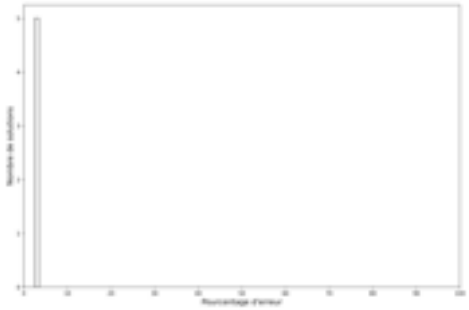

Figure 10 – Local Search

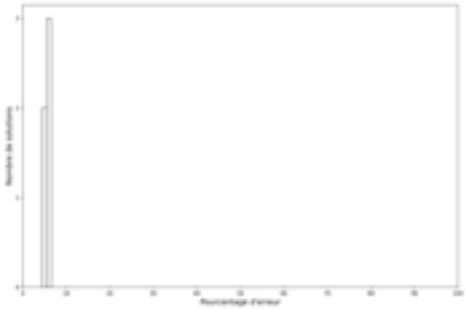

Figure 11 – Matching Based

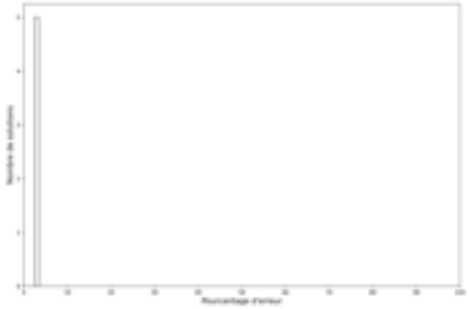

Figure 12 – Maximum Degree Greedy

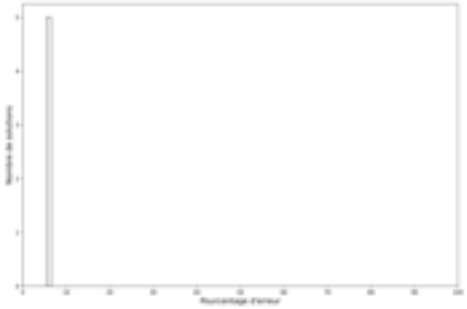

Figure 13 – Minimum Degree Greedy

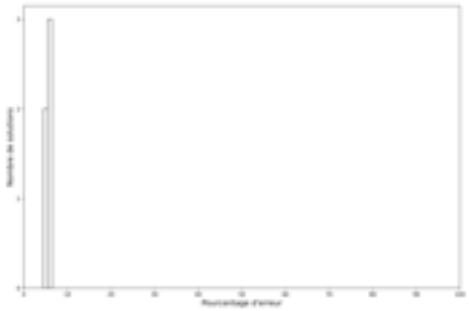

Figure 14 – Primal Dual

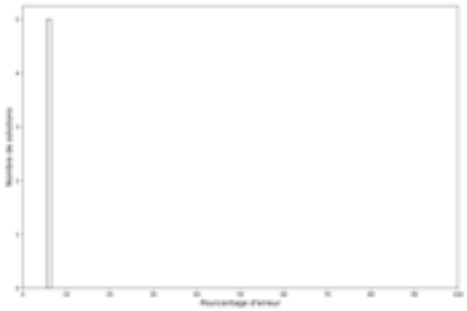

Figure 15 – Sorted ListRight

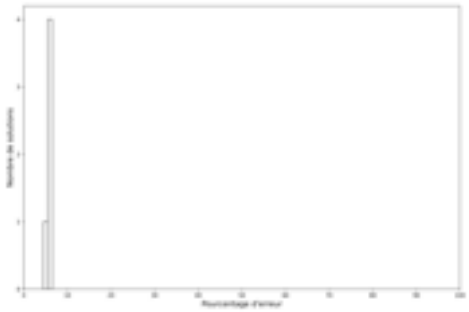

Figure 16 – Sorted ListLeft

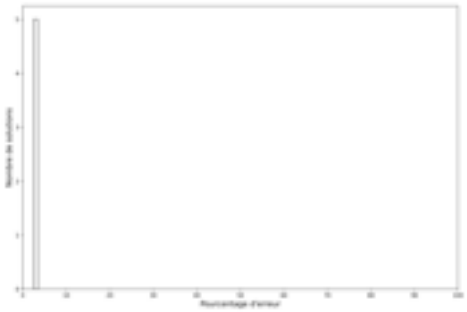

Figure 17 – VNS

Échantillons du benchmark BHOSLIB

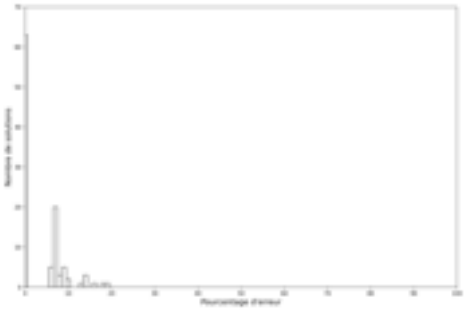

Figure 18 – ACO

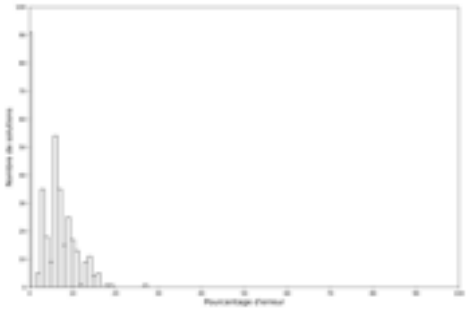

Figure 19 – Approximate Matching

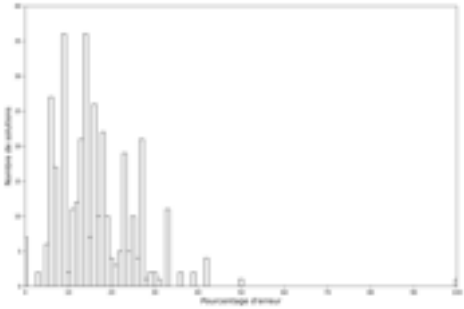

Figure 20 – Bar Yehuda Even

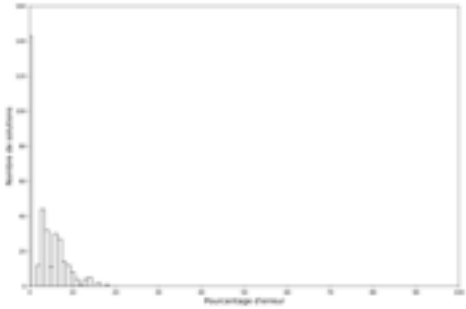

Figure 21 – Degree Reduction

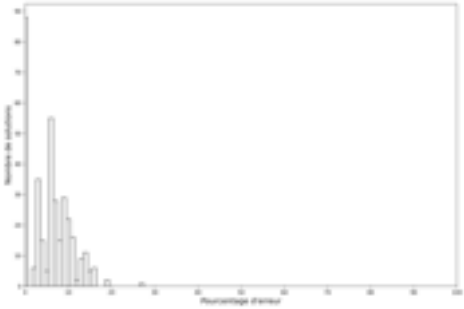

Figure 22 – Edge Deletion

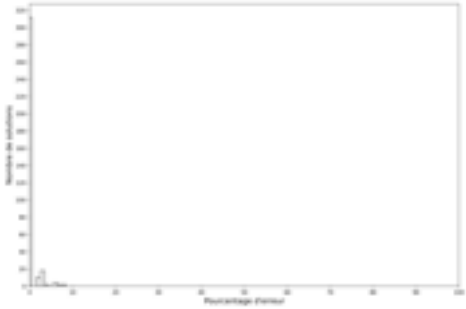

Figure 23 – Greedy Independant Cover

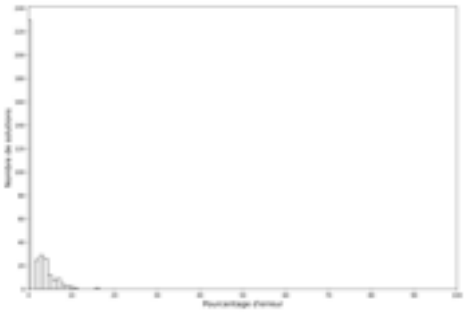

Figure 24 – ILS

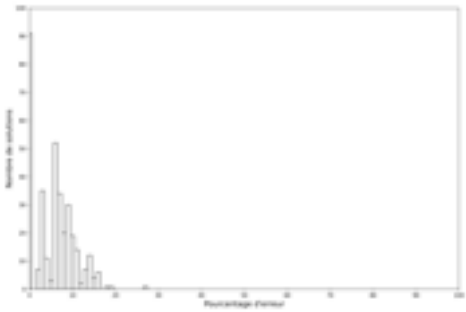

Figure 25 – Local Search Ratio

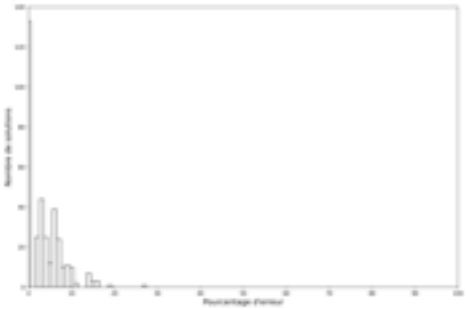

Figure 26 – Local Search

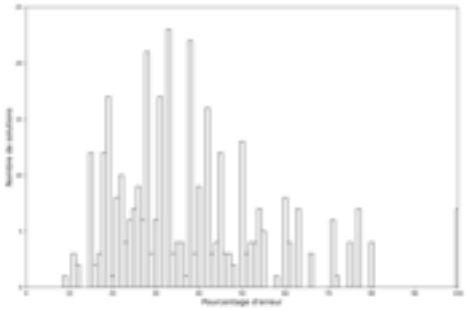

Figure 27 – Matching Based

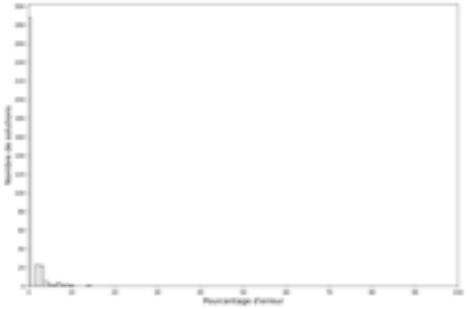

Figure 28 – Maximum Degree Greedy

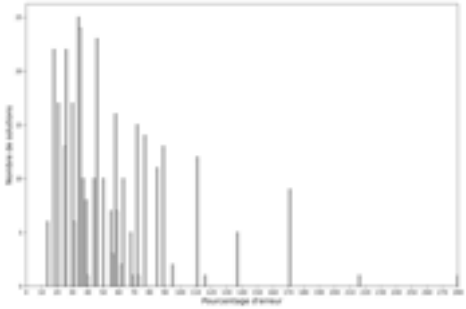

Figure 29 – Minimum Degree Greedy

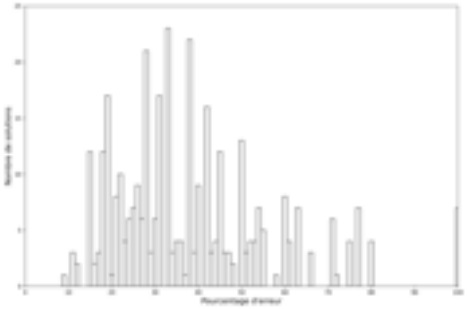

Figure 30 – Primal Dual

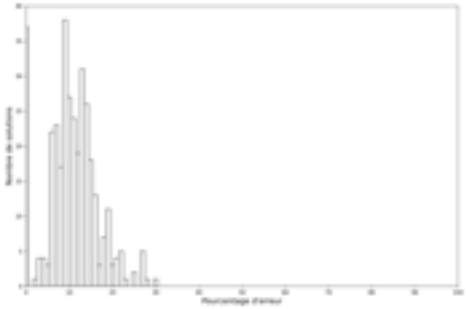

Figure 31 – Sorted ListLeft

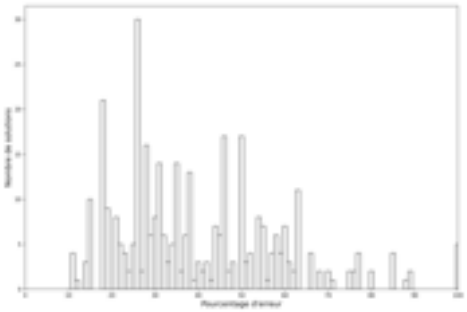

Figure 32 – Sorted ListRight

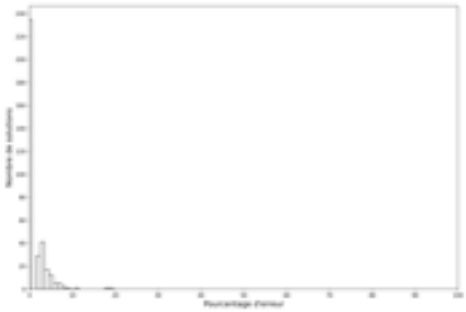

Figure 33 – VNS

Échantillons des graphes de Erdös-Renyi

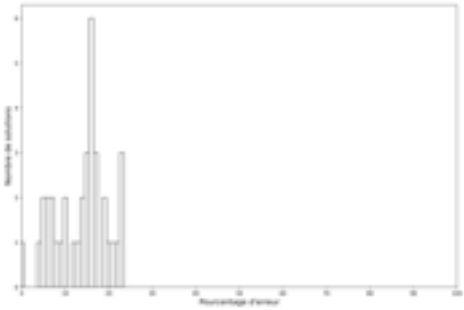

Figure 34 – ACO

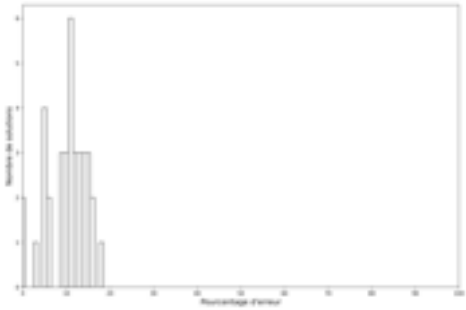

Figure 35 – Approximate Matching

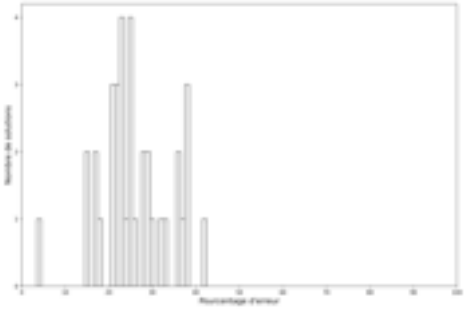

Figure 36 – Bar Yehuda Even

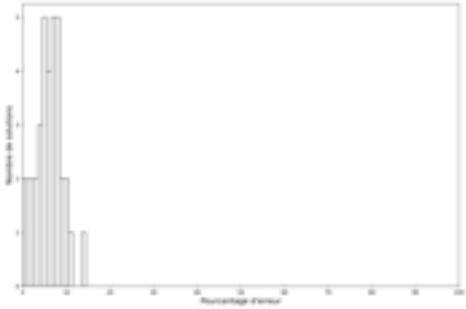

Figure 37 – Degree Reduction

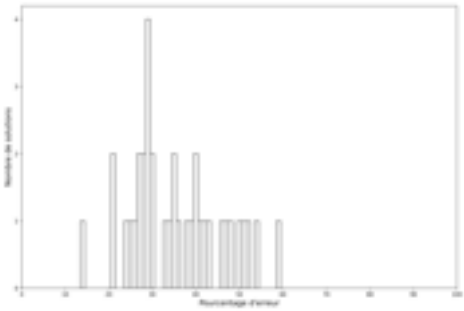

Figure 38 – DFS

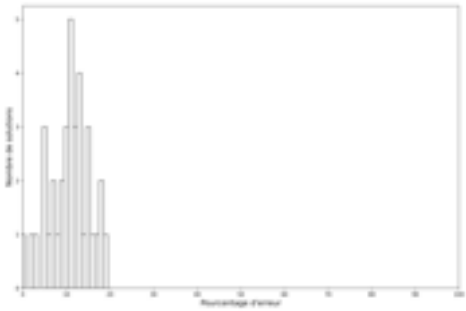

Figure 39 – Edge Deletion

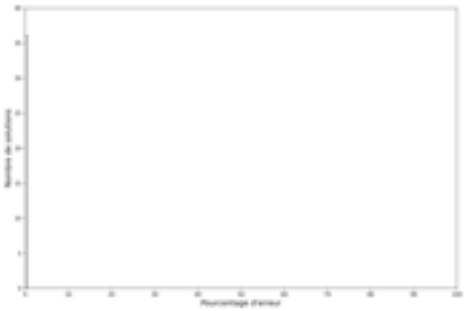

Figure 40 – Greedy Independant Cover

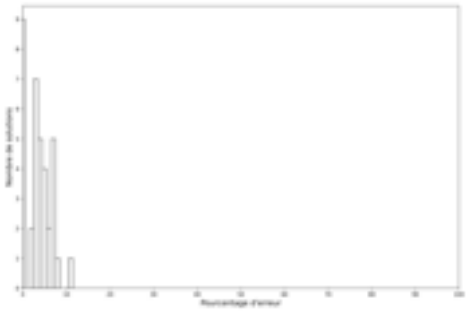

Figure 41 – ILS

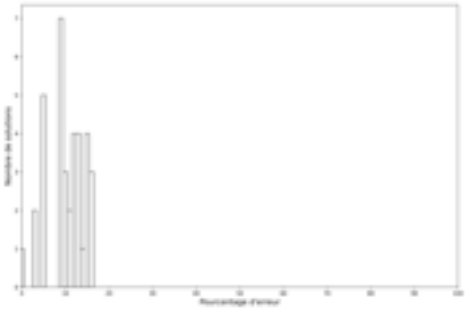

FIGURE 42 – Local Search Ratio

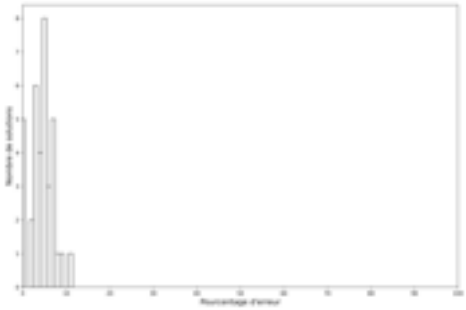

FIGURE 43 – Local Search

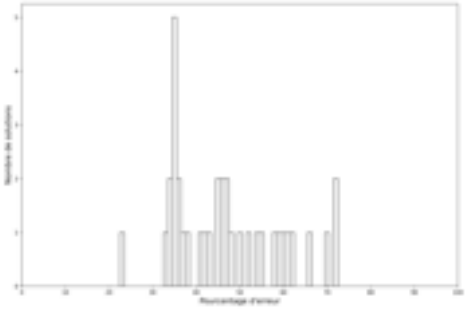

FIGURE 44 – Matching Based

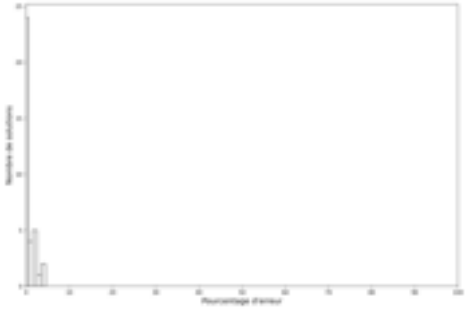

FIGURE 45 – Maximum Degree Greedy

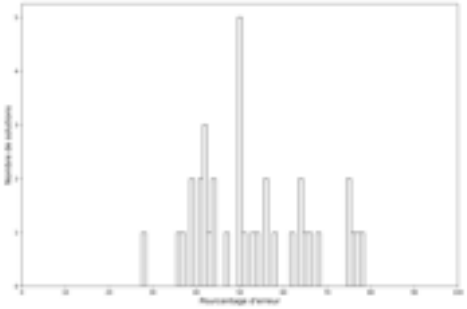

FIGURE 46 – Minimum Degree Greedy

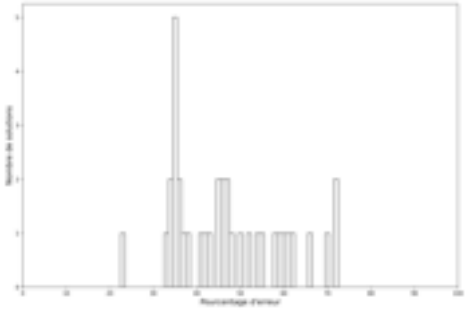

FIGURE 47 – Primal Dual

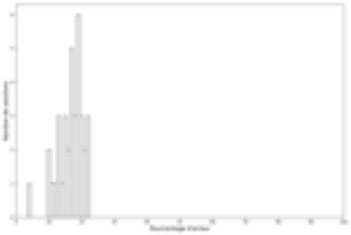

FIGURE 48 – Sorted ListRight

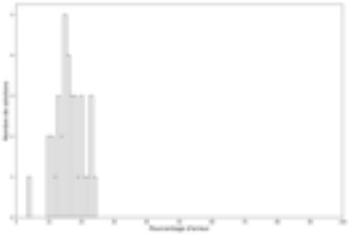

FIGURE 49 – Sorted LsitLeft

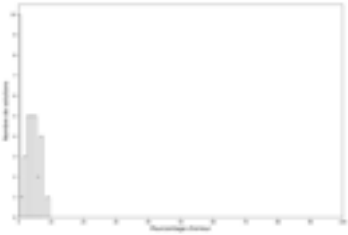

FIGURE 50 – VNS

Échantillons des graphes régulier

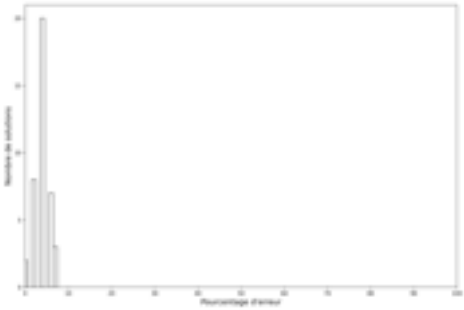

Figure 51 – ACO

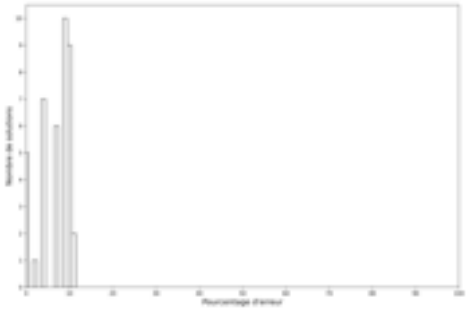

Figure 52 – Approximate Matching

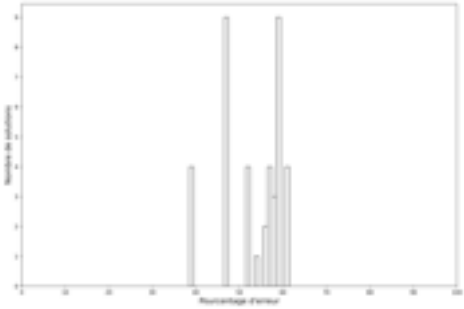

Figure 53 – Bar Yehuda Even

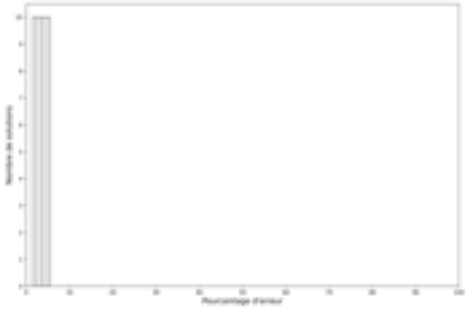

Figure 54 – Degree Reduction

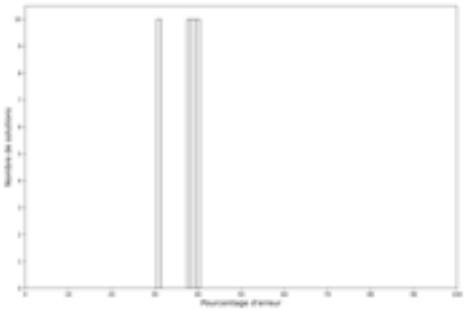

Figure 55 – DFS

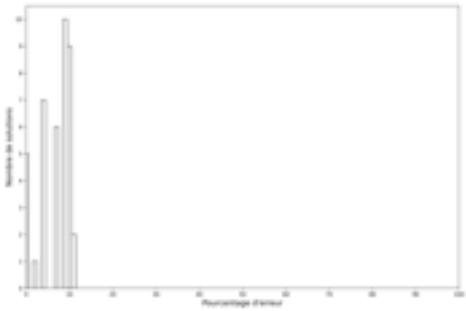

Figure 56 – Edge Deletion

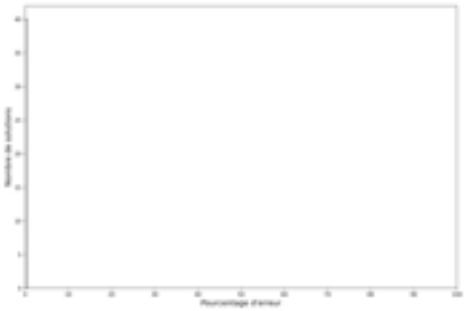

Figure 57 – Greedy Independant Cover

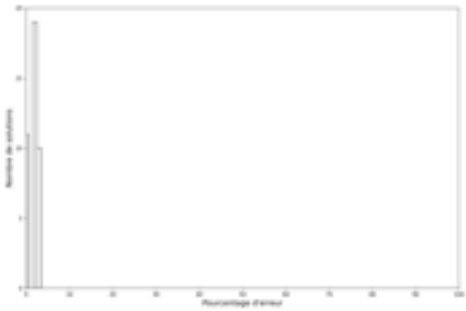

Figure 58 – ILS

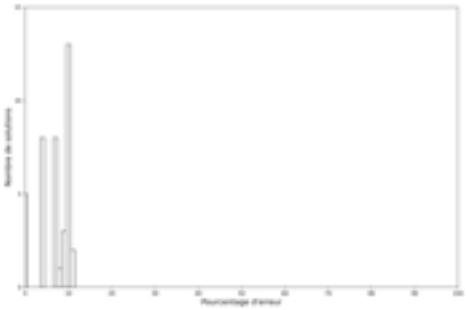

FIGURE 59 – Local Search Ratio

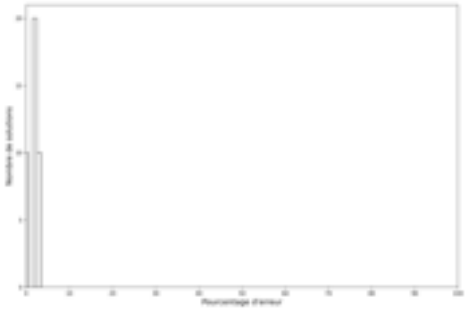

FIGURE 60 – Local Search

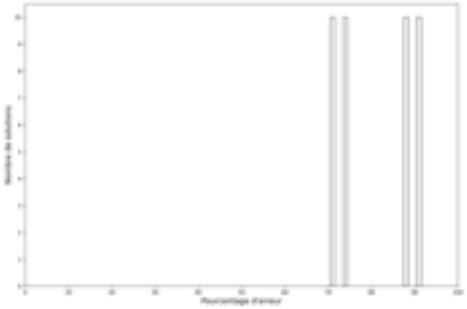

FIGURE 61 – Matching Based

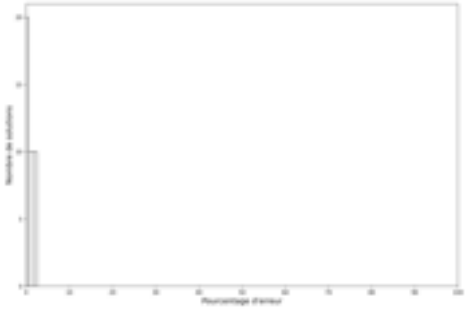

FIGURE 62 – Maximum Degree Greedy

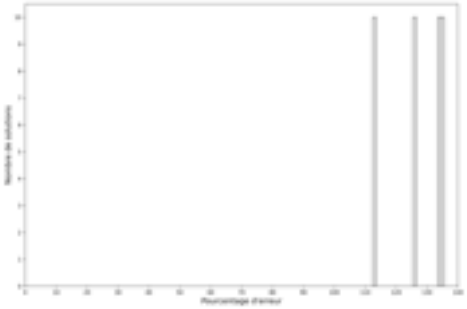

FIGURE 63 – Minimum Degree Greedy

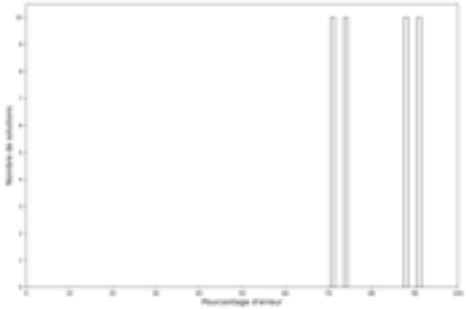

FIGURE 64 – Primal Dual

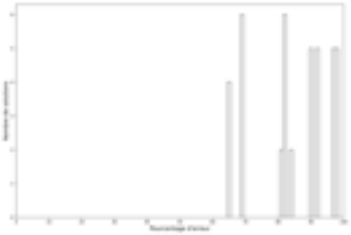

FIGURE 65 – Sorted ListRight

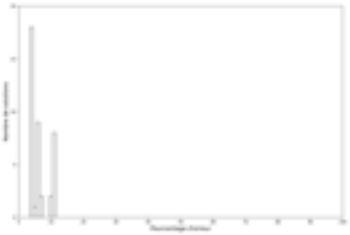

FIGURE 66 – Sorted ListLeft

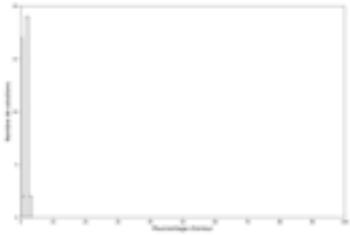

FIGURE 67 – VNS

Échantillons des arbres

Annexe .5 : Rapport d'approximation en moyen cas

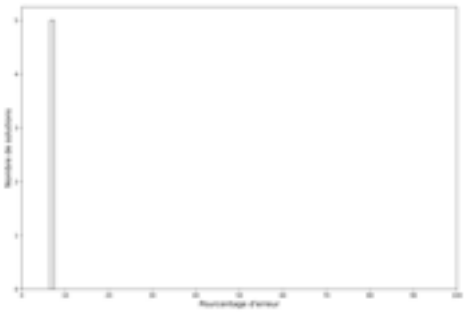

Figure 68 – ACO

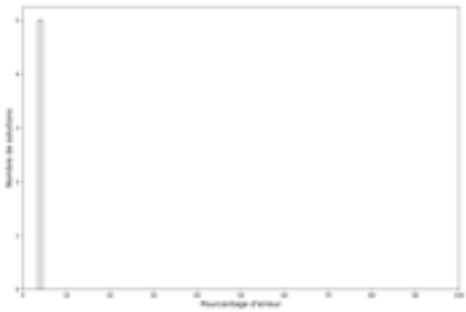

Figure 69 – Approximate Matching

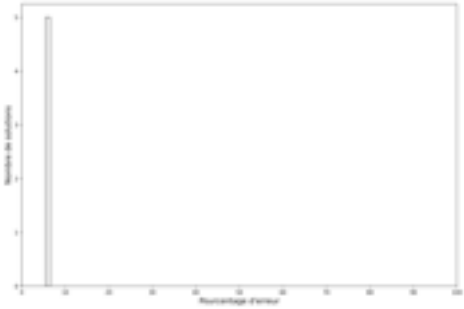

Figure 70 – Bar Yehuda Even

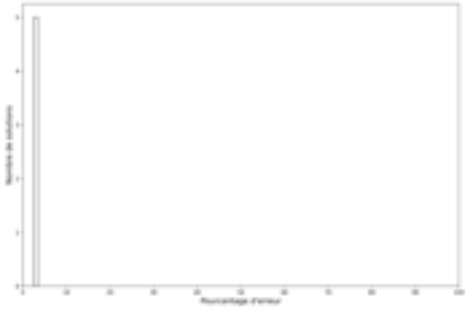

Figure 71 – Degree Reduction

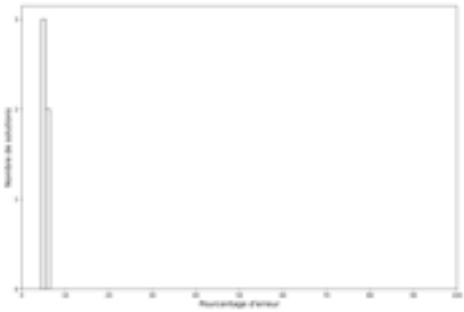

Figure 72 – DFS

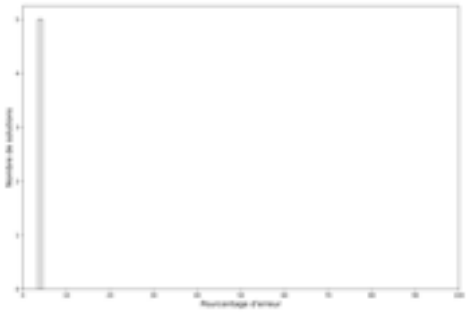

Figure 73 – Edge Deletion

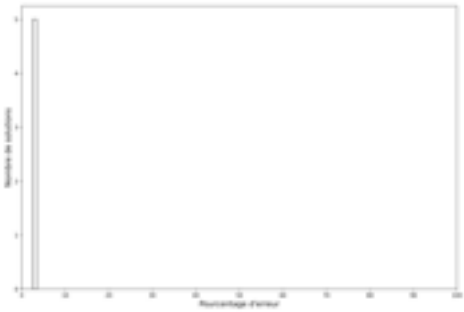

Figure 74 – Greedy Independant Cover

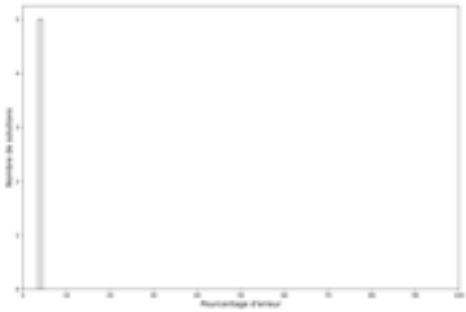

Figure 75 – Local Search Ratio

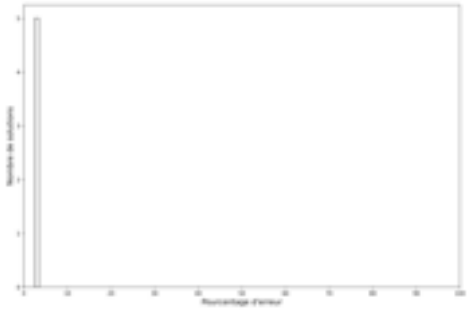

Figure 76 – Local Search

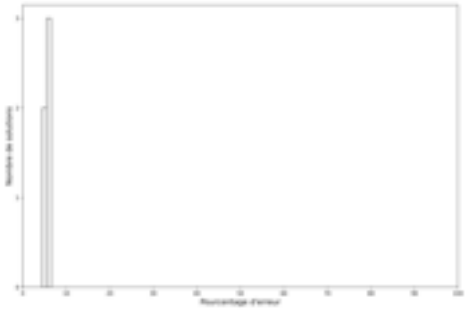

Figure 77 – Matching Based

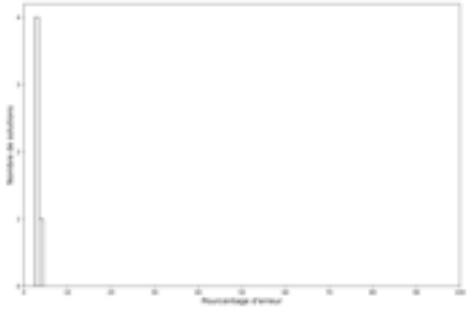

Figure 78 – Maximum Degree Greedy

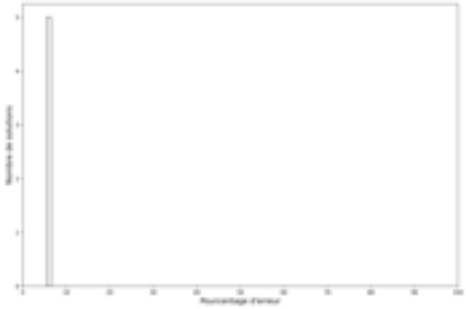

Figure 79 – Minimum Degree Greedy

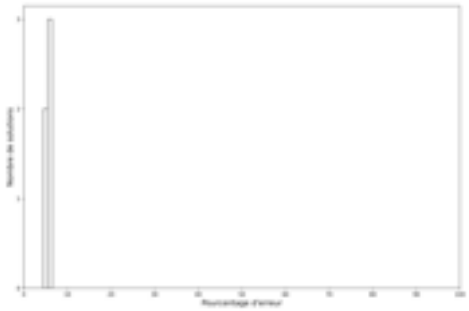

Figure 80 – Primal Dual

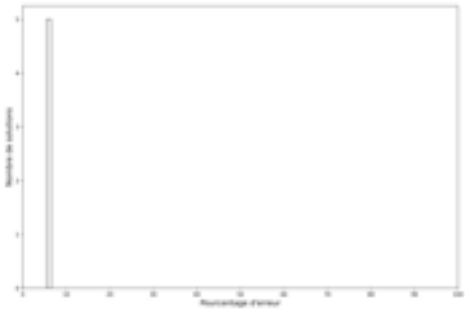

Figure 81 – Sorted ListRight

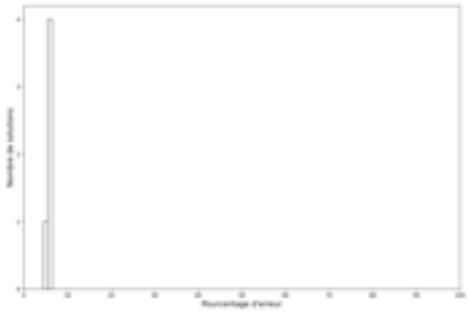

Figure 82 – Sorted ListLeft

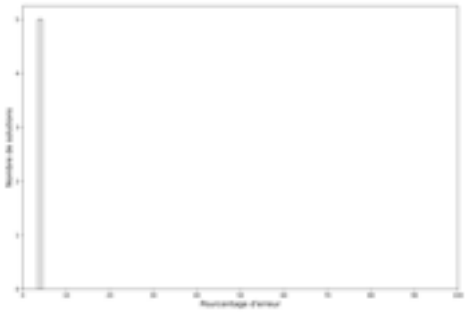

Figure 83 – VNS

Échantillons du benchmark BHOSLIB

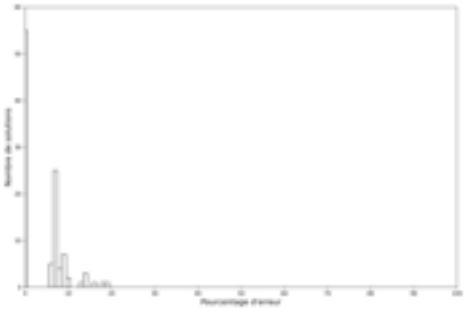

Figure 84 – ACO

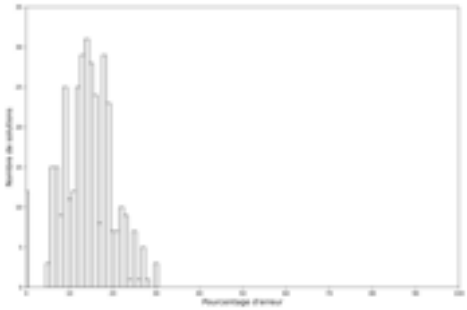

Figure 85 – Approximate Matching

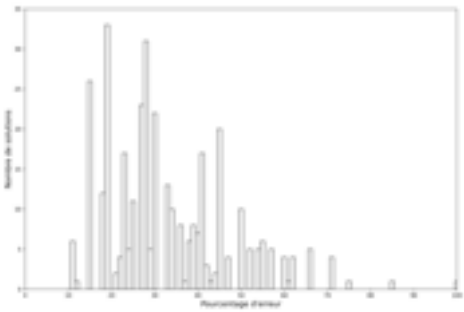

Figure 86 – Bar Yehuda Even

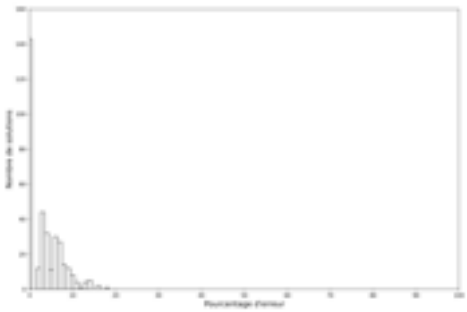

Figure 87 – Degree Reduction

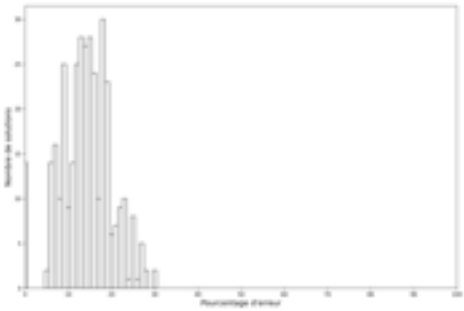

Figure 88 – Edge Deletion

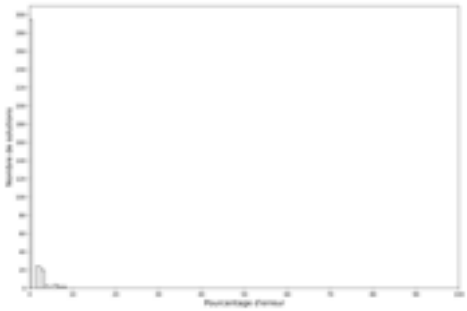

Figure 89 – Greedy Independant Cover

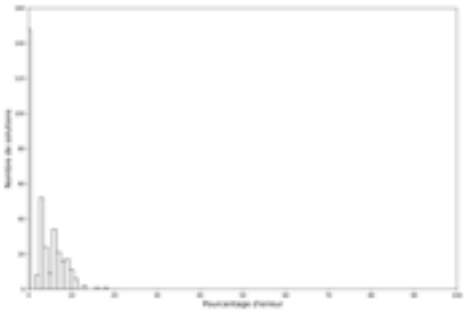

Figure 90 – ILS

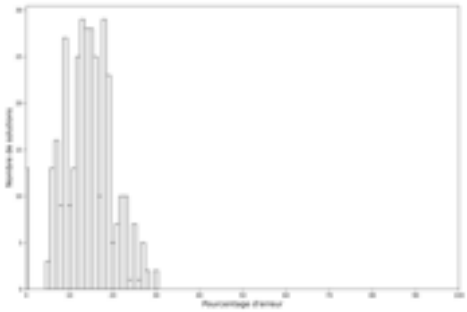

Figure 91 – Local Search Ratio

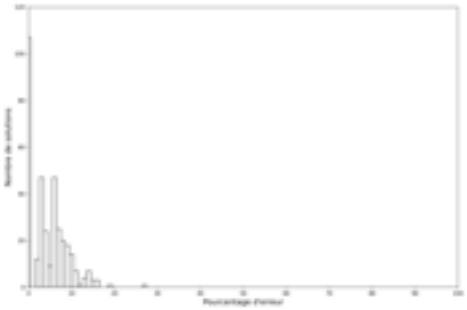

Figure 92 – Local Search

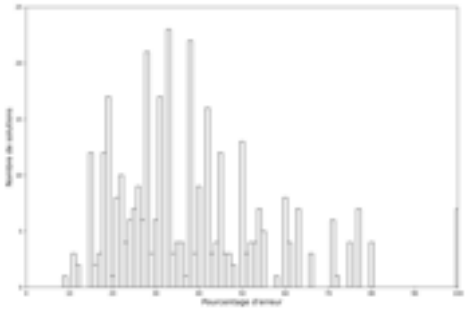

Figure 93 – Matching Based

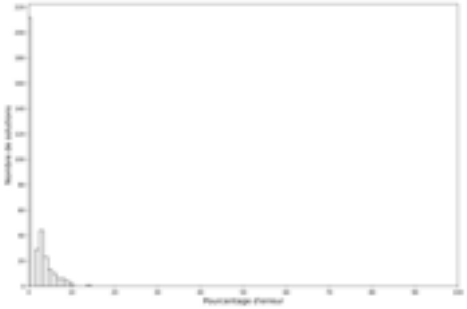

Figure 94 – Maximum Degree Greedy

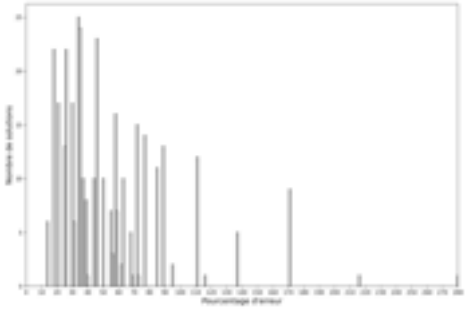

Figure 95 – Minimum Degree Greedy

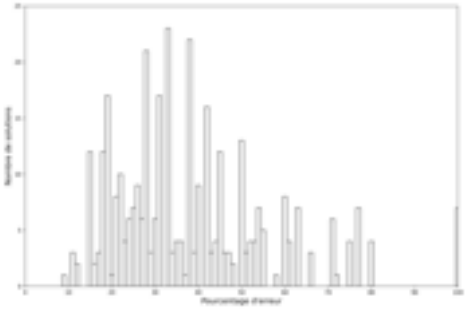

Figure 96 – Primal Dual

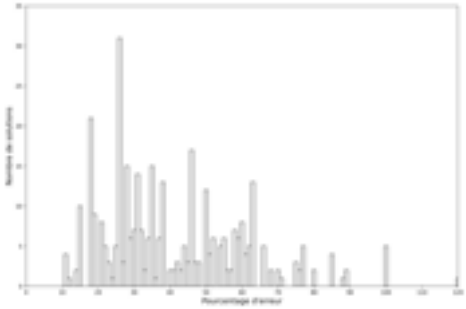

Figure 97 – Sorted ListRight

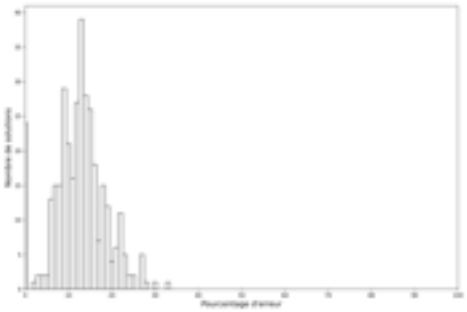

Figure 98 – Sorted ListLeft

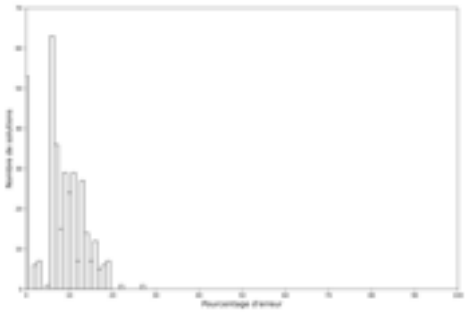

Figure 99 – VNS

Échantillons des graphes de Erdös-Renyi

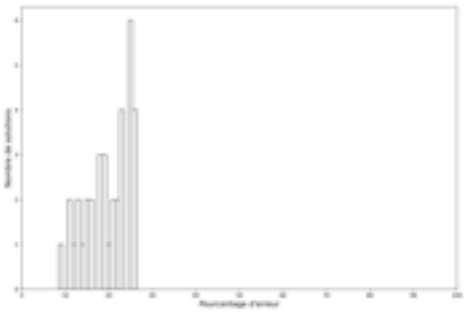

Figure 100 – ACO

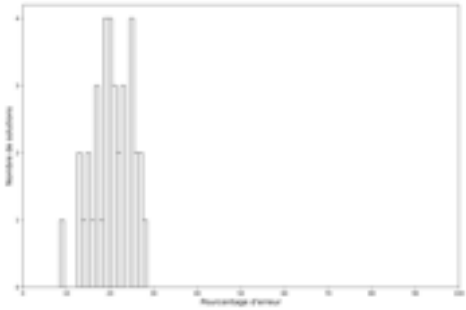

Figure 101 – Approximate Matching

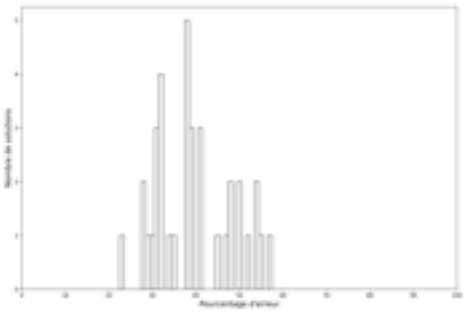

Figure 102 – Bar Yehuda Even

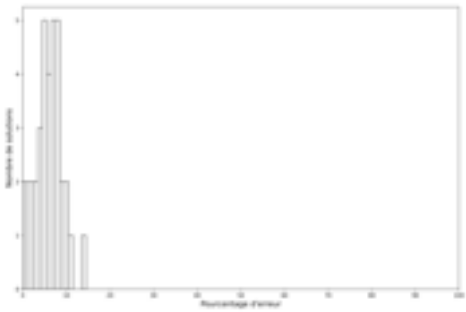

Figure 103 – Degree Reduction

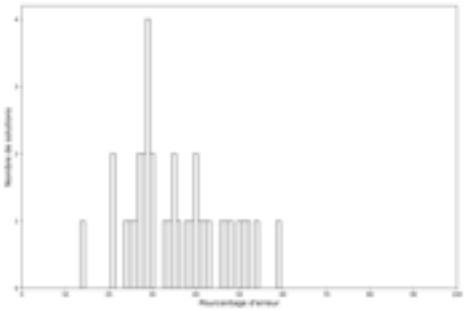

Figure 104 – DFS

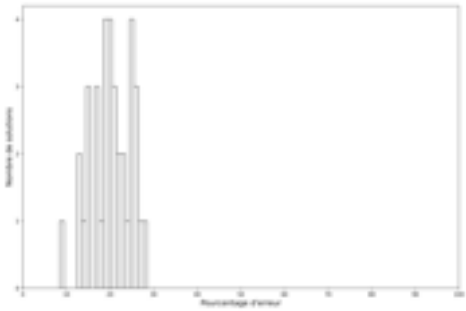

Figure 105 – Edge Deletion

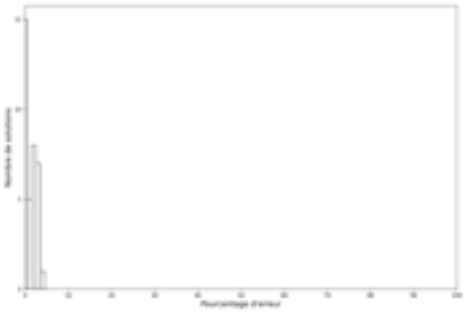

Figure 106 – Greedy Independant Cover

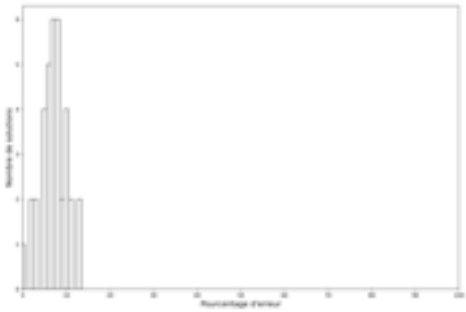

Figure 107 – ILS

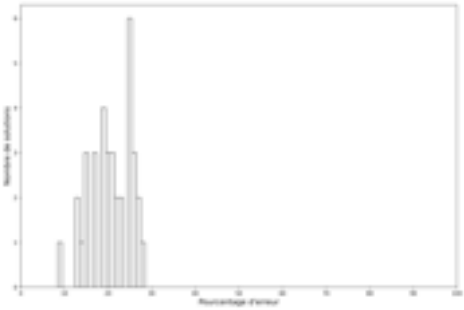

FIGURE 108 – Local Search Ratio

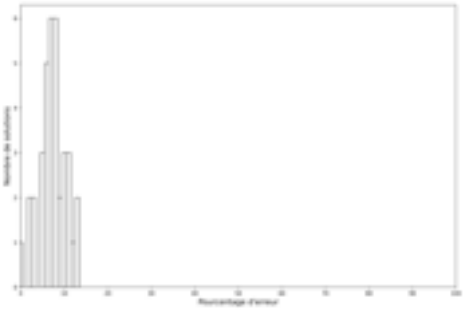

FIGURE 109 – Local Search

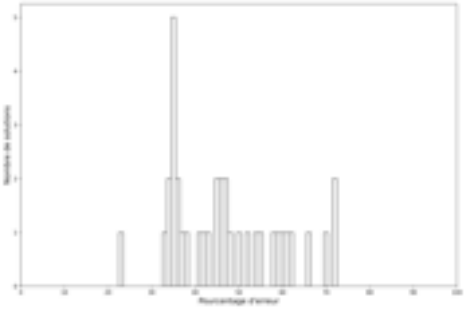

FIGURE 110 – Matching Based

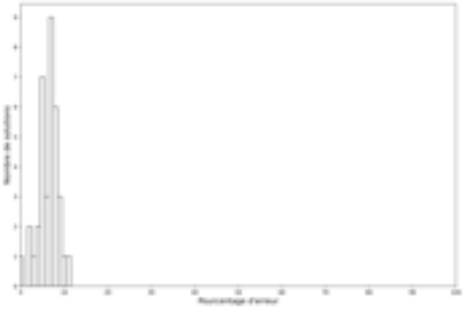

FIGURE 111 – Maximum Degree Greedy

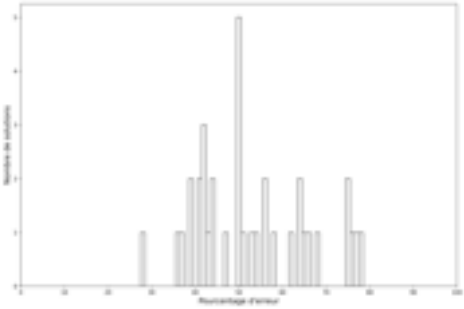

FIGURE 112 – Minimum Degree Greedy

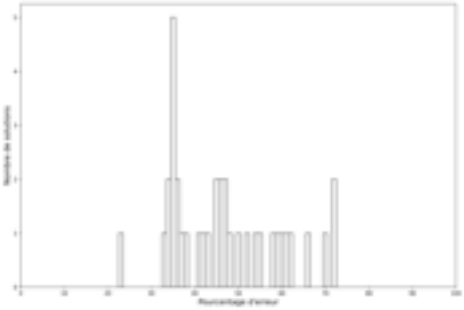

FIGURE 113 – Primal Dual

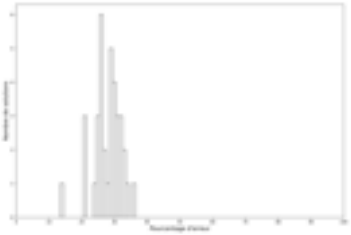

FIGURE 114 – Sorted ListRight

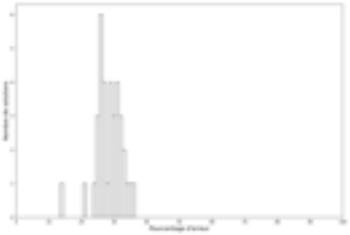

FIGURE 115 – Sorted ListLeft

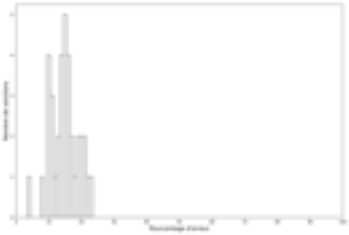

FIGURE 116 – VNS

Échantillons des graphes régulier

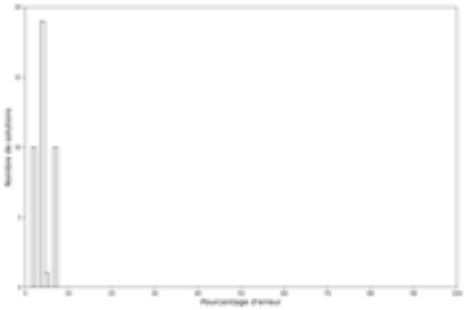

Figure 117 – ACO

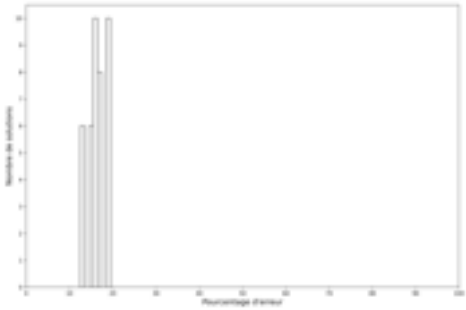

Figure 118 – Approximate Matching

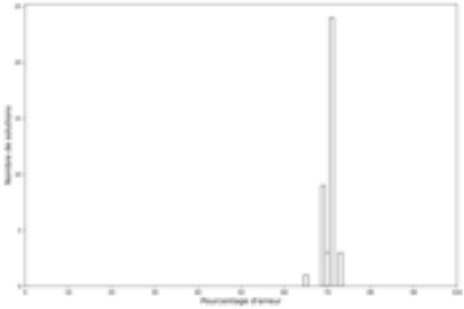

Figure 119 – Bar Yehuda Even

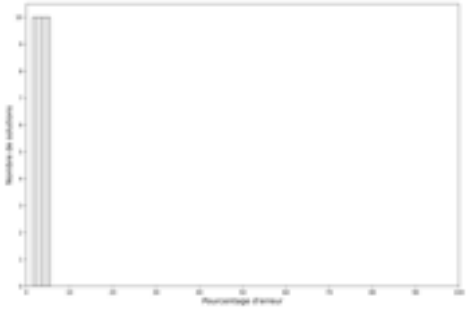

Figure 120 – Degree Reduction

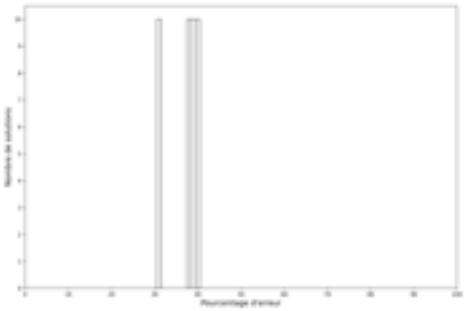

Figure 121 – DFS

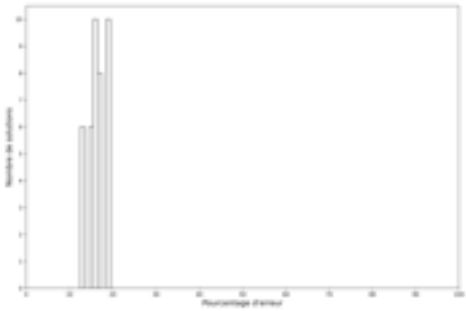

Figure 122 – Edge Deletion

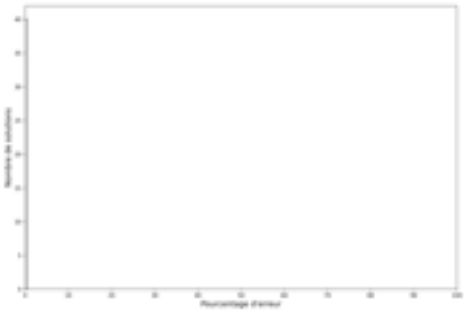

Figure 123 – Greedy Independant Cover

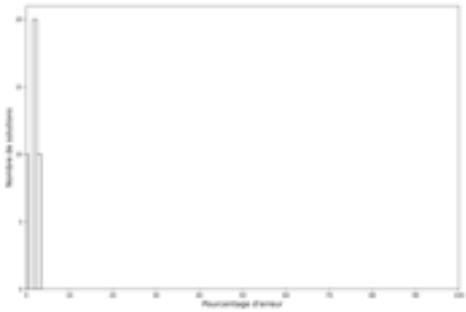

Figure 124 – ILS

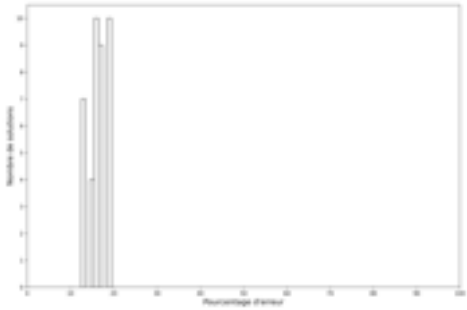

Figure 125 – Local Search Ratio

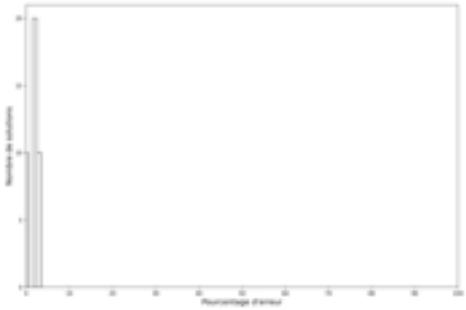

Figure 126 – Local Search

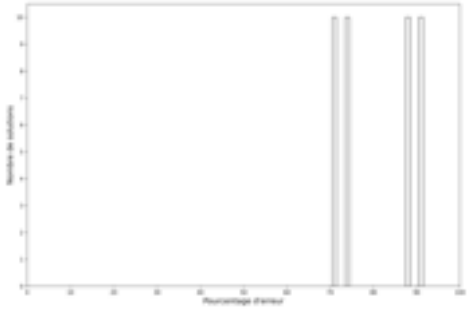

Figure 127 – Matching Based

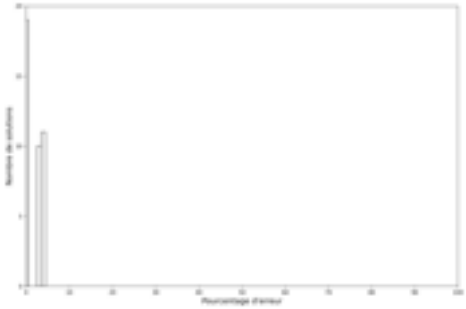

Figure 128 – Maximum Degree Greedy

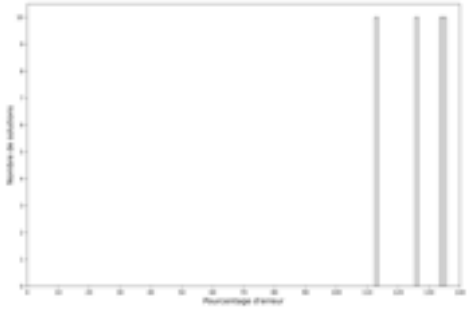

Figure 129 – Minimum Degree Greedy

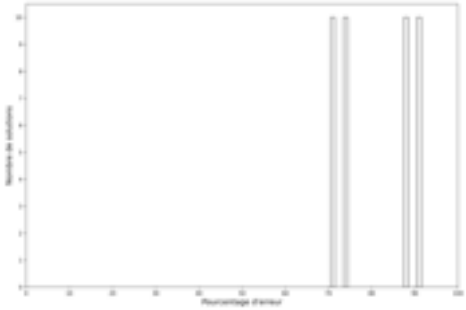

Figure 130 – Primal Dual

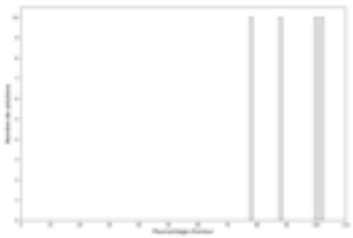

Figure 131 – Sorted ListRight

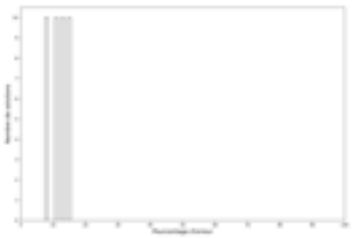

Figure 132 – Sorted ListLeft

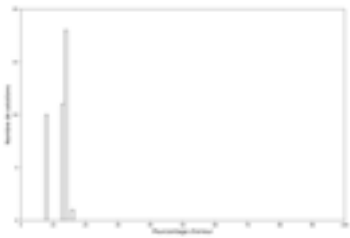

Figure 133 – VNS

Échantillons des arbres

Annexe .6 : Rapport d'approximation en pire cas

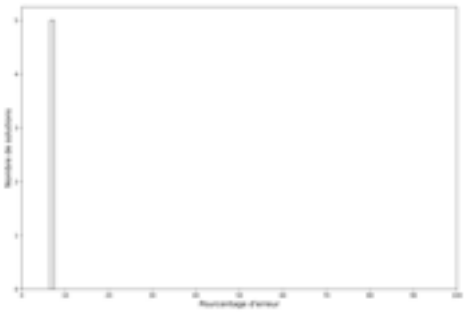

Figure 134 – ACO

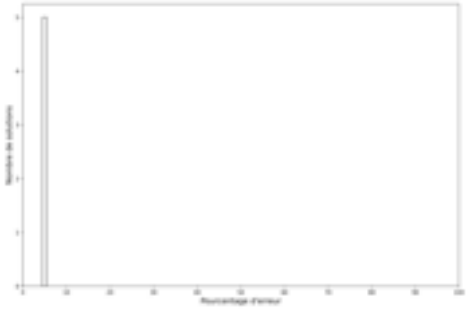

Figure 135 – Approximate Matching

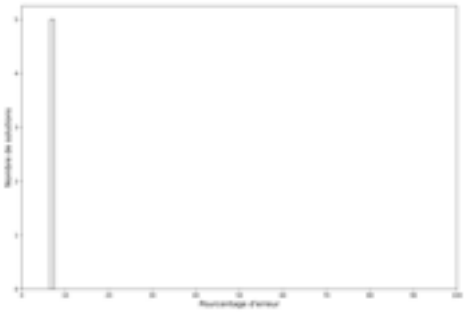

Figure 136 – Bar Yehuda Even

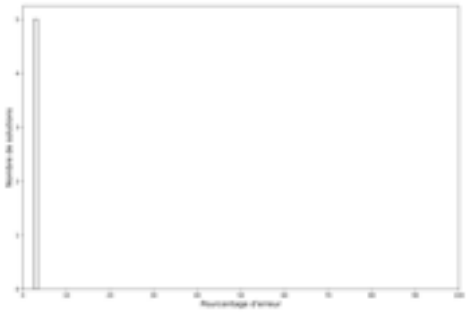

Figure 137 – Degree Reduction

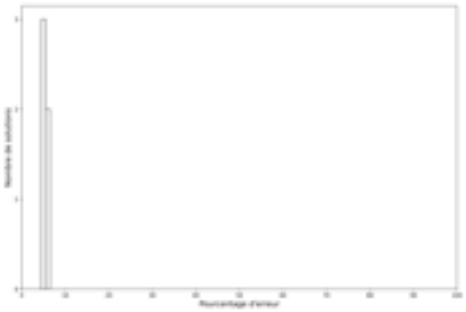

Figure 138 – DFS

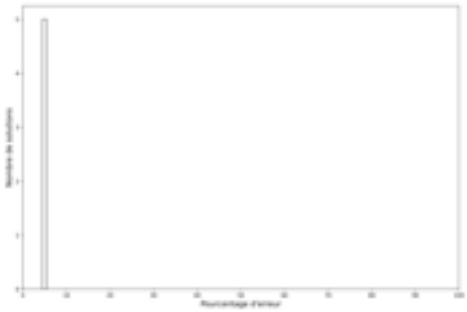

Figure 139 – Edge Deletion

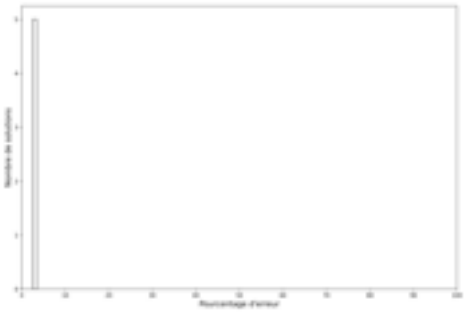

Figure 140 – Greedy Independant Cover

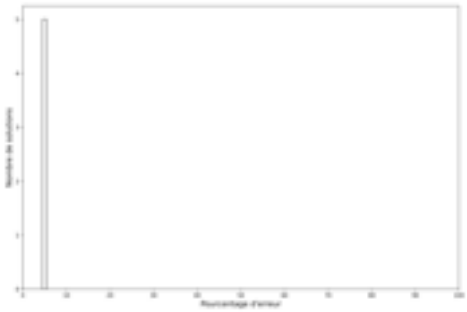

Figure 141 – Local Search Ratio

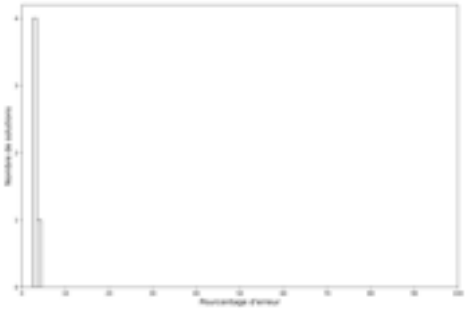

Figure 142 – Local Search

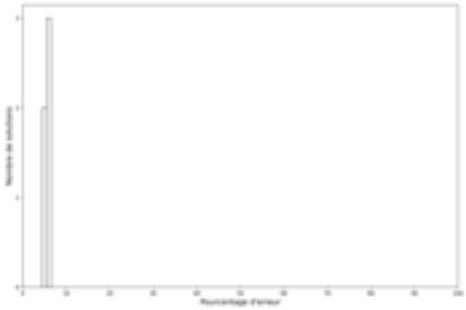

Figure 143 – Local Search

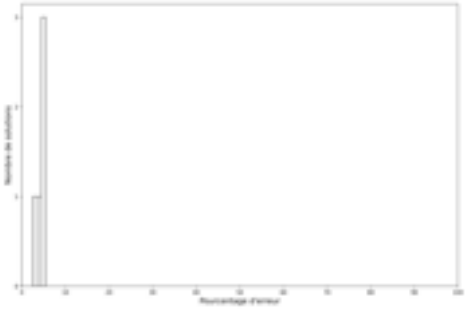

Figure 144 – Maximum Degree Greedy

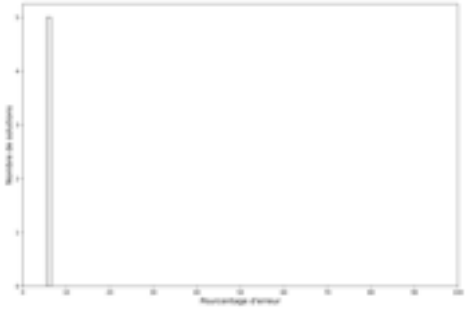

Figure 145 – Minimum Degree Greedy

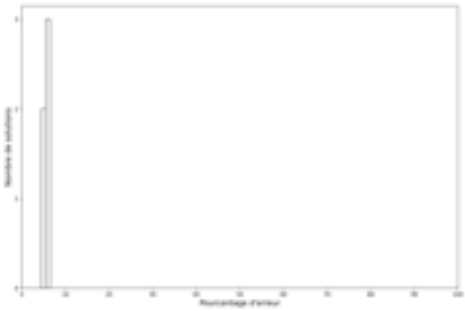

Figure 146 – Primal Dual

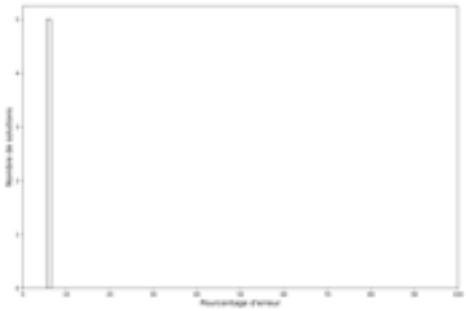

Figure 147 – Sorted ListRight

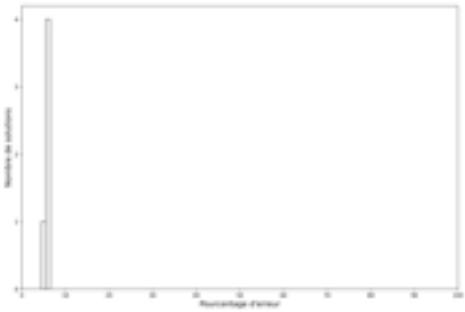

Figure 148 – Sorted ListLeft

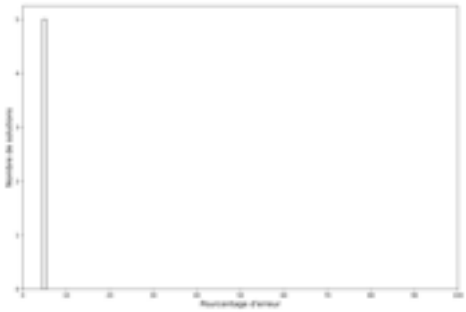

Figure 149 – VNS

Échantillons du benchmark BHOSLIB

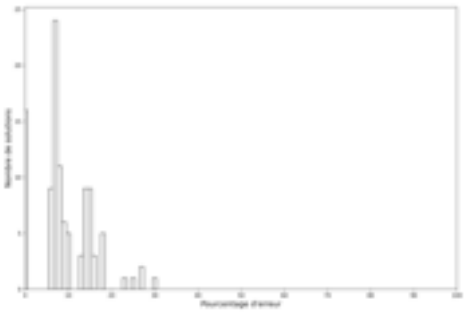

Figure 150 – ACO

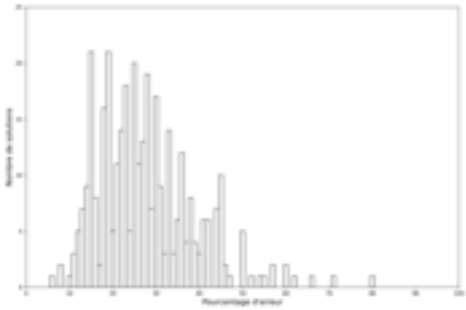

Figure 151 – Approximate Matching

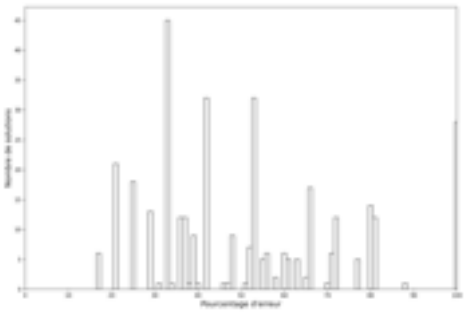

Figure 152 – Bar Yehuda Even

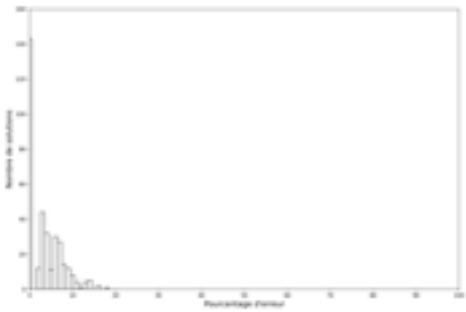

Figure 153 – Degree Reduction

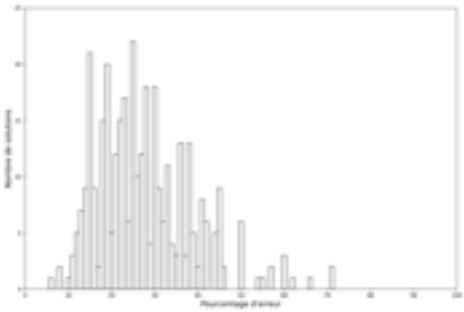

Figure 154 – Edge Deletion

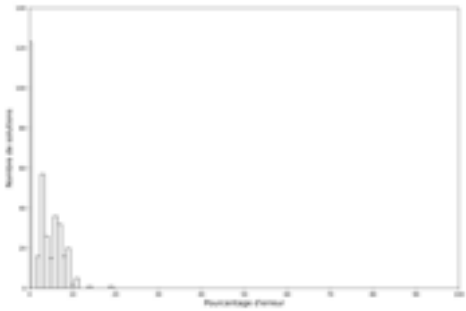

Figure 155 – Greedy Independant Cover

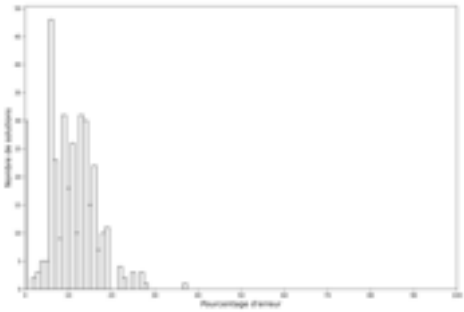

Figure 156 – ILS

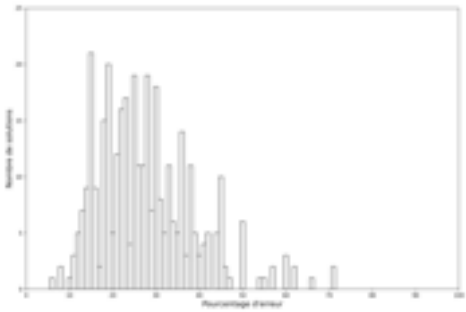

Figure 157 – Local Search Ratio

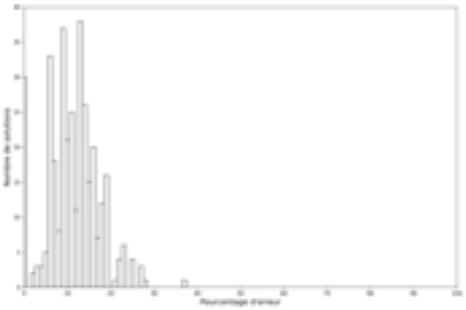

Figure 158 – Local Search

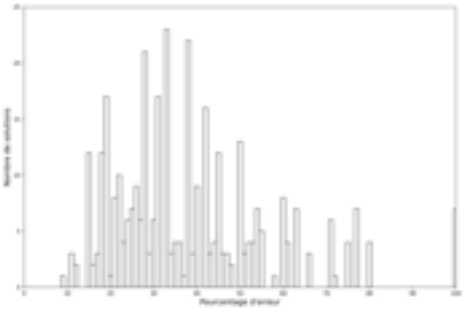

Figure 159 – Matching Based

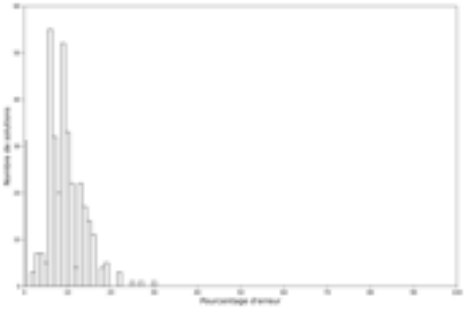

Figure 160 – Maximum Degree Greedy

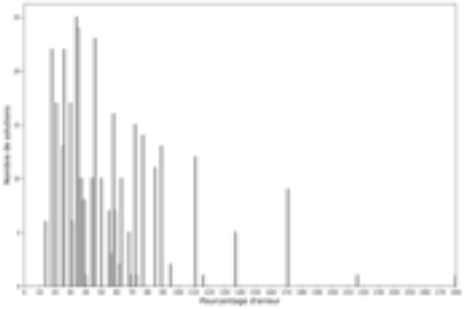

Figure 161 – Minimum Degree Greedy

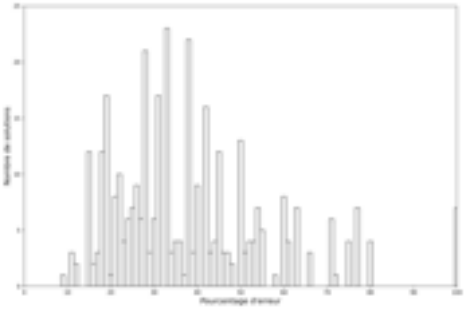

Figure 162 – Primal Dual

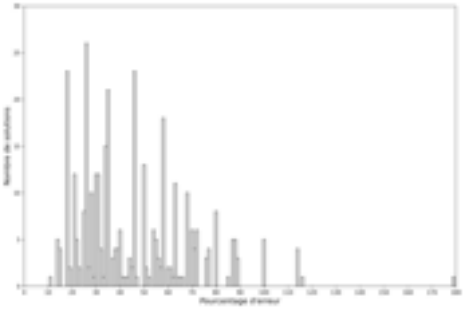

Figure 163 – Sorted ListRight

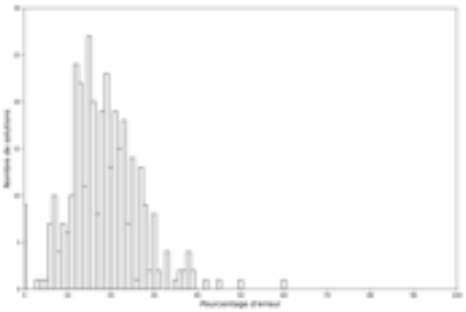

Figure 164 – Sorted ListLeft

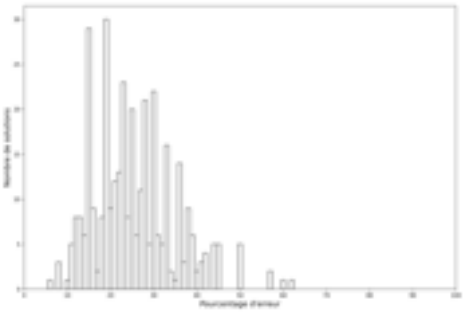

Figure 165 – VNS

Échantillons des graphes de Erdös-Renyi

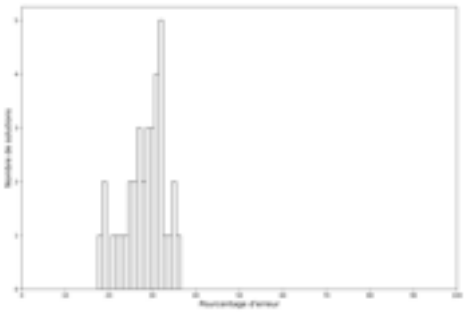

Figure 166 – ACO

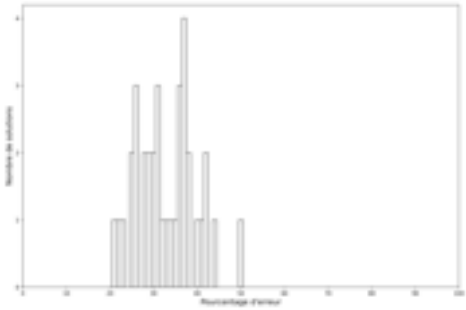

Figure 167 – Approximate Matching

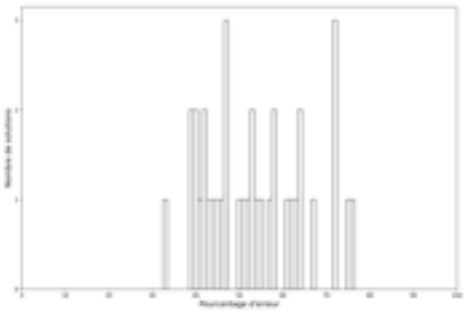

Figure 168 – Bar Yehuda Even

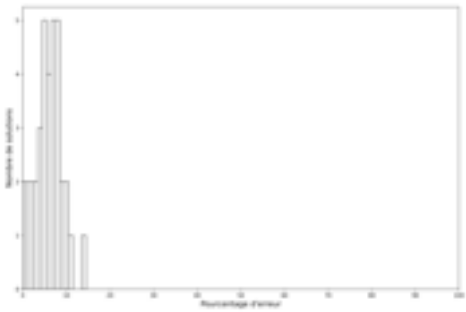

Figure 169 – Degree Reduction

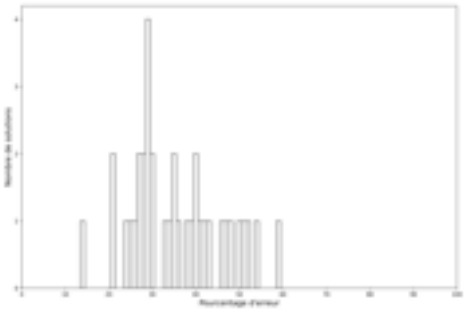

Figure 170 – DFS

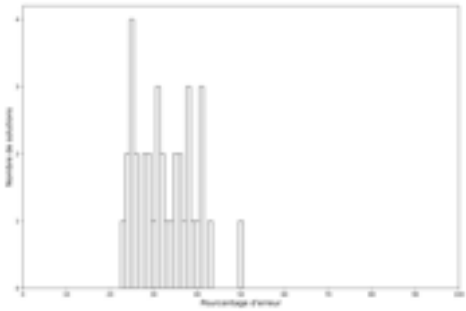

Figure 171 – Edge Deletion

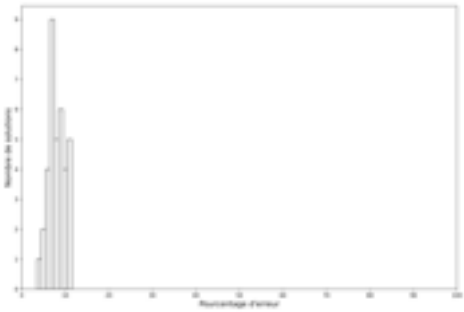

Figure 172 – Greedy Independant Cover

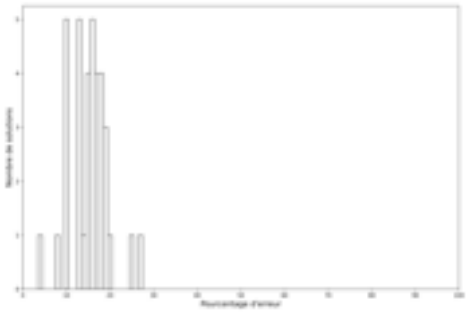

Figure 173 – ILS

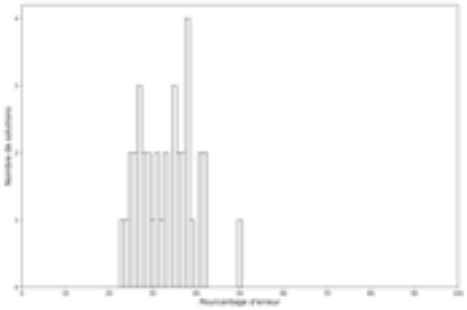

FIGURE 174 – Local Search Ratio

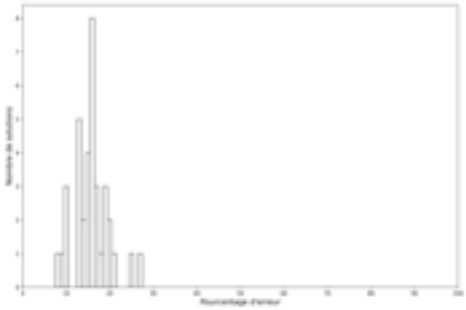

FIGURE 175 – Local Search

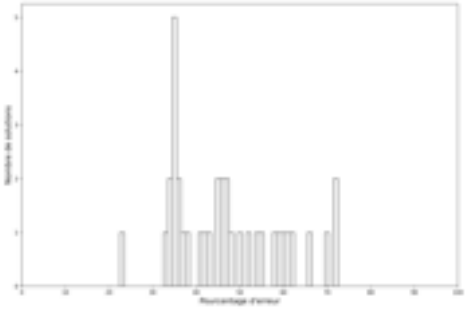

FIGURE 176 – Matching Based

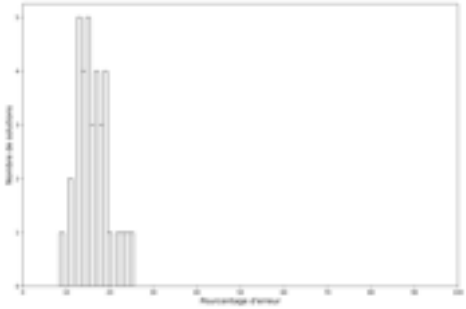

FIGURE 177 – Maximum Degree Greedy

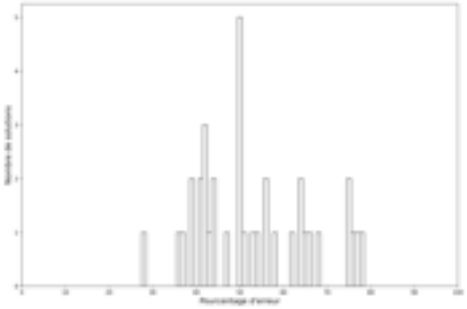

FIGURE 178 – Minimum Degree Greedy

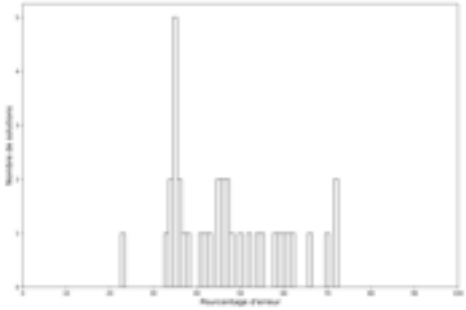

FIGURE 179 – Primal Dua

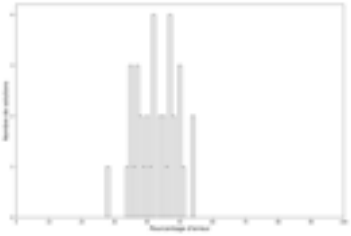

FIGURE 180 – Sorted ListRight

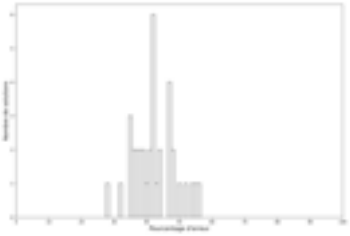

FIGURE 181 – Sorted ListLeft

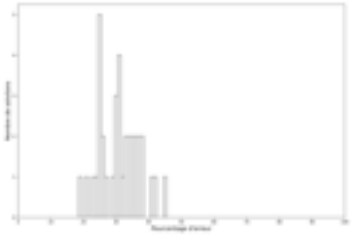

FIGURE 182 – VNS

Échantillons des graphes régulier

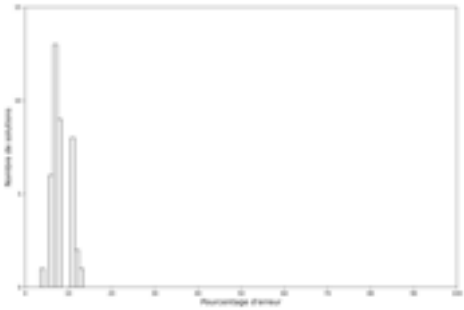

Figure 183 – ACO

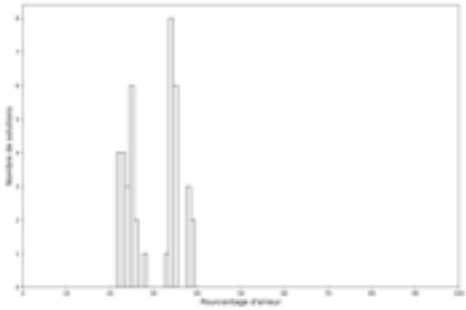

Figure 184 – Approximate Matching

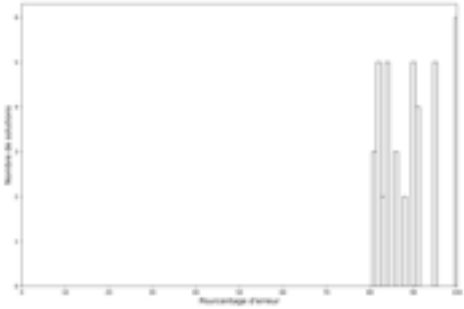

Figure 185 – Bar Yehuda Even

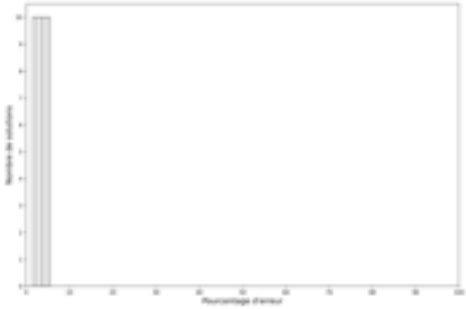

Figure 186 – Degree Reduction

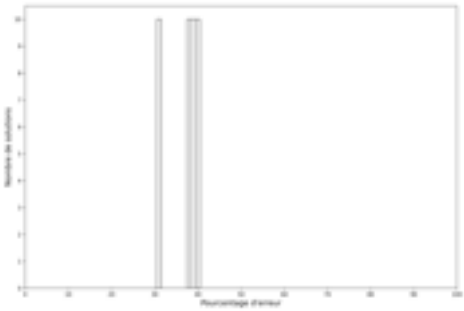

Figure 187 – DFS

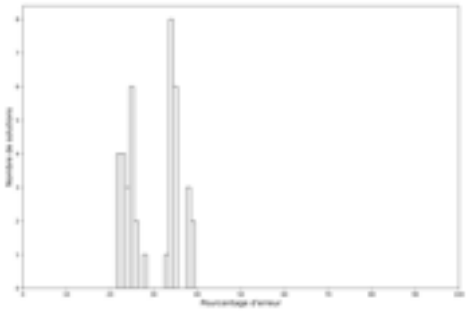

Figure 188 – Edge Deletion

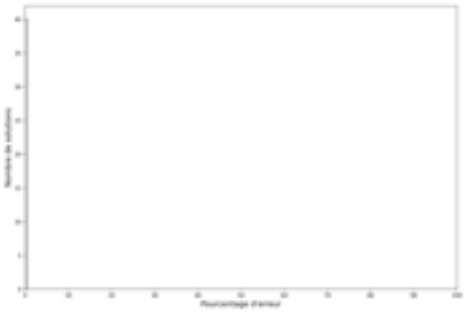

Figure 189 – Greedy Independant Cover

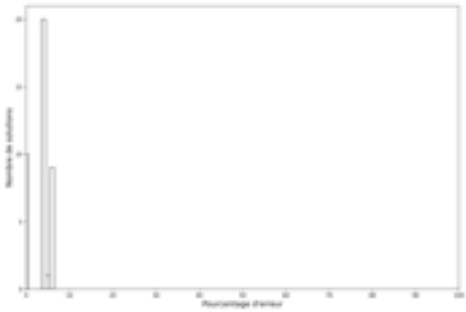

Figure 190 – ILS

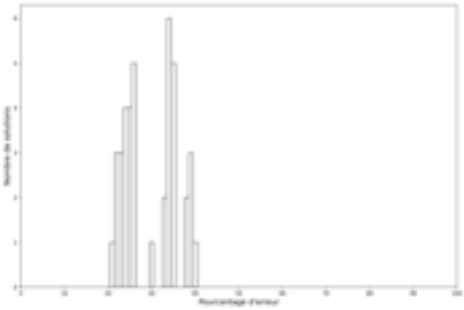

FIGURE 191 – Local Search Ratio

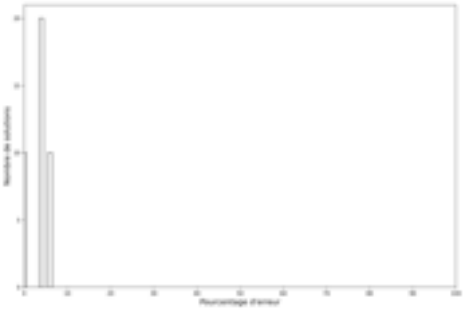

FIGURE 192 – Local Search

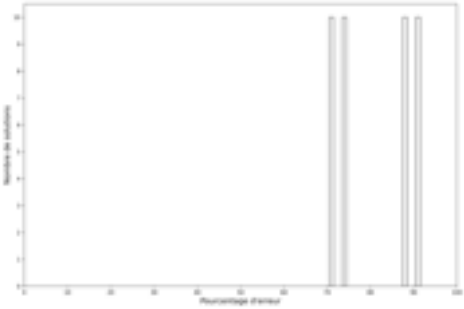

FIGURE 193 – Matching Based

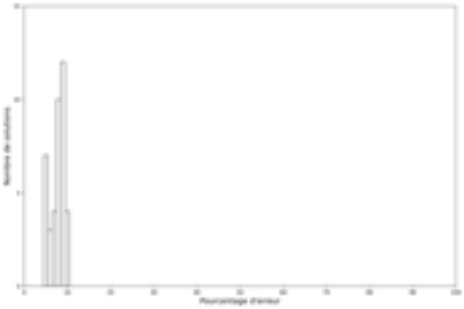

FIGURE 194 – Maximum Degree Greedy

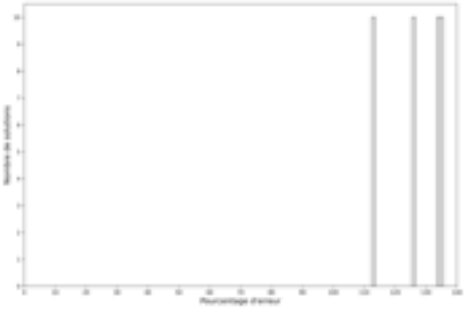

FIGURE 195 – Minimum Degree Greedy

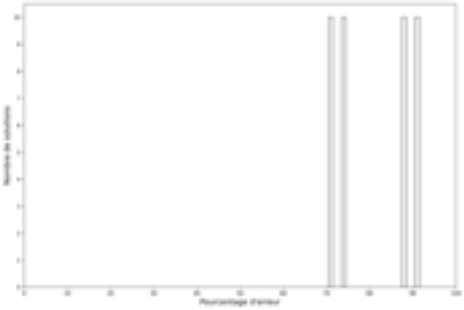

FIGURE 196 – Primal Dual

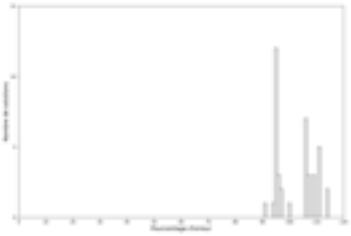

FIGURE 197 – Sorted ListRight

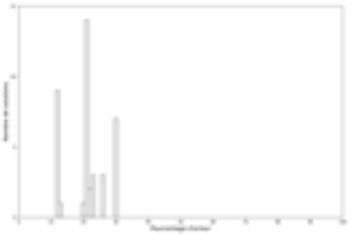

FIGURE 198 – Sorted ListLeft

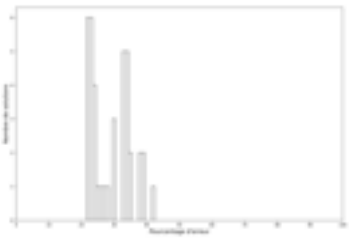

FIGURE 199 – VNS

Échantillons des arbres

Annexe .7 : Une autre annexe